

END OF STUDIES PROJECT REPORT

By

Youssef HASNAOUI

Design and Implementation of a Debugger with Instruction Trace Capabilities

Professional Supervisor:

Mohamed HAMROUNI

Academic Supervisor:

Faten Salem

Project Elaborated Within STMicroelectronics



Dedications

I dedicate this work to my family and friends.

*To my mother **Monia Sellami**, your love and support means so much to me in every step of my journey. You're the best.*

*To my father **Makram Hasnaoui**, thank you so much for all the sacrifices you've made for my sake. I could never be here without you.*

*To my sister **Aicha**, your kindness and your belief in me have been a constant source of inspiration, and for that, I am very grateful.*

To my friends, you have my most sincere gratitude for your support, your guidance and your company throughout this journey.

Thank you,

Youssef HASNAOUI

Acknowledgements

I would like to sincerely thank everyone who had a hand in the development of this project.

Firstly, I want to express my utmost appreciation to **Mohamed HAMROUNI**, his guidance and support have been invaluable, and have kept me motivated and determined to give my best effort.

I would also like to thank my professor **Faten SALEM** for providing the essential foundation to pursue this domain of expertise and for her captivating curiosity, which inspired me to continuously expand the boundaries of my knowledge.

Last but not least, to every single person that contributed to the successful completion of this project with their constant support and motivation, you have my utmost thanks.

Contents

List of Figures	viii
List of Tables	ix
List of Acronyms	xi
Introduction	1
1 Context & Problem Statement	3
1.1 Host Organization	3
1.1.1 STMicroelectronics	3
1.1.2 ST Tunis	3
1.1.3 Application & Product Support Team	3
1.2 Observing an Executing Program	4
1.2.1 State at a Halt	5
1.2.2 The Cost of Instrumentation	5
1.2.3 Defects Requiring Execution History	6
1.3 Problem Statement	6
1.4 State of the Art: Instruction Trace on ARM Microcontrollers	7
1.4.1 Trace Capability of the STM32 Devices	7
1.4.2 Proprietary Development Environments	8
1.4.3 Open-Source Debug Software	9
1.4.4 Published Work	9
1.4.5 Positioning of this Work	9
1.5 Objectives, Constraints and Deliverables	10
1.5.1 Constraints	10
1.5.2 Deliverables	11
1.6 Conclusion	11
2 Foundations of Debugging and Instruction Trace	12
2.1 Debugging Fundamentals	12
2.1.1 Anatomy of a Debugger	12
2.1.2 The Debug Adapter Protocol	13

2.1.3	The GDB Remote Serial Protocol	13
2.2	Debug Primitives	13
2.2.1	Run Control	14
2.2.2	Breakpoints: Software and Hardware	14
2.2.3	State Inspection: Registers, Memory and Variables	14
2.2.4	Call Stacks and Unwinding	15
2.2.5	Disassembly	15
2.3	Program Representation	15
2.3.1	The Embedded Build Model	15
2.3.2	The ELF Format	16
2.3.3	DWARF Debug Information	16
2.4	Microcontrollers	17
2.4.1	Device Description	17
2.5	The ARM Cortex-M Architecture	18
2.5.1	AMBA Buses and their Roles in Debug	18
2.6	The ARM Debug Architecture	19
2.6.1	The ARM Debug Interface	19
2.6.2	Debug Port: JTAG and SWD	19
2.6.3	Access Ports and the Memory Access Port	20
2.6.4	Component Identification	20
2.6.5	Reset Types and Vector Catch	20
2.6.6	Breakpoint and Watchpoint Units	20
2.7	The CoreSight Trace Architecture	20
2.7.1	Trace Sources, Links and Sinks	21
2.7.2	Instruction, Data and Instrumentation Trace	21
2.7.3	Trace Source: the Embedded Trace Macrocell	21
2.7.4	The CoreSight Trace Formatter	24
2.7.5	Off-Chip Trace Sinks: TPIU and SWO	25
2.7.6	On-Chip Trace Sinks: the Trace Memory Controller	25
2.8	Conclusion	26
3	Requirements Specification and Work Environment	28
3.1	Scope of the Deliverables	28
3.2	Requirements Analysis	29

3.2.1	Functional Requirements	29
3.2.2	Non-Functional Requirements	30
3.2.3	Traceability to Deliverables	30
3.3	Hardware Environment	30
3.3.1	Microcontroller: STM32H7S3L8	30
3.3.2	Development Board: NUCLEO-H7S3L8	31
3.3.3	Debug Probe: STLINK-V3EC	31
3.3.4	Trace Resources Available on the Target	31
3.4	Software Environment	32
3.4.1	Development Environment	32
3.4.2	External Software Components	32
3.5	Conclusion	33
4	Architecture	34
4.1	System Overview	34
4.1.1	Global Architecture	34
4.1.2	Debugger Capabilities	35
4.1.3	Design Rules	35
4.2	Debugger Architecture	35
4.2.1	Transport Layer	36
4.2.2	The Event Bus	36
4.2.3	Orchestration Layer	36
4.2.4	Request Handlers	37
4.2.5	The Debug Context	37
4.2.6	State Managers	38
4.2.7	External-System Providers	38
4.2.8	Concurrency Model	39
4.3	Instruction Trace Architecture	39
4.3.1	Pipeline Stages	39
4.3.2	Trace Data Model	40
4.3.3	Trace Lifecycle	40
4.4	Frontend Architecture	41
4.5	Conclusion	41

5	Implementation	42
5.1	Development Methodology	42
5.2	The Application Note	43
5.3	On-Chip Debugger: OpenOCD	43
5.3.1	Embedding as a Library	43
5.3.2	CoreSight Drivers	45
5.3.3	Trace Extraction	46
5.4	Source-Level Debugging through LLDB	47
5.5	Division of Work	48
5.6	The Trace Pipeline	49
5.6.1	Decoding	49
5.6.2	Static Program Analysis	50
5.6.3	Reconstruction	51
5.6.4	Delivery	51
5.7	The Debugger	51
5.7.1	Session Lifecycle	51
5.7.2	Reset and Resynchronization	52
5.7.3	Breakpoints	53
5.7.4	Memory and Observation	53
5.7.5	Peripherals	54
5.7.6	Variables and Expressions	54
5.7.7	Protocol Extensions	55
5.8	The Frontend	55
5.8.1	Portability and Provenance	57
5.9	Conclusion	57
6	Experimental Validation and Results	58
6.1	How the Engine Was Tested	58
6.2	Hardware Test Bench	59
6.3	Tests That Run Without the Board	59
6.3.1	Protocol Handshake and Cancellation	60
6.3.2	Event Delivery From Several Threads	60
6.3.3	Static Program Analysis	60
6.3.4	Trace Decoding and Reconstruction	61

6.3.5	Parsing the Device Description	62
6.3.6	Two Copies of LLVM in One Process	62
6.4	Tests That Run on the Board	62
6.4.1	Reading Memory Through the Right Access Port	62
6.4.2	Trace-Unit Registers and Capture Delivery	62
6.4.3	Launch and the Live Trace Path	63
6.4.4	Stepping	63
6.4.5	Peripherals and Live Watching	63
6.4.6	Reset and Resynchronization	64
6.5	Tests Through the Extension	64
6.5.1	View Logic	64
6.5.2	End to End on the Board	64
6.6	Comparison Against Other Tools	68
6.7	Defects Found on Hardware	68
6.8	Results	68
6.9	Limitations and Future Work	70
6.10	Conclusion	71
	Conclusion	72
	Bibliography	73
	Annexes	76
	Annexe 1. Registered Protocol Requests	76
	Annexe 2. Test Inventory	77

List of Figures

2.1	Conceptual decomposition of a debugger	13
2.2	Software and hardware breakpoint mechanisms	14
2.3	Structure of a Debug Access Port	19
2.4	Trace elements generated for a short loop	23
2.5	Structure of a formatter frame	24
3.1	Debug and trace infrastructure of the target	31
4.1	Global architecture of the solution	34
4.2	Module dependency graph	36
4.3	Request flow from transport to response	37
4.4	The debug context	38
4.5	The four-stage instruction trace pipeline	40
4.6	The trace lifecycle	41
5.1	The development methodology	43
5.2	Command queue and wake mechanism	45
5.3	TMC state transitions	45
5.4	Extraction sequence	47
5.5	Division of work between LLDB and OpenOCD	49
5.6	From bytes to elements	50
5.7	Reset and resynchronization	52
5.8	Breakpoint placement decision	53
5.9	Peripheral write and observation	54
5.10	The peripheral view	56
5.11	The live memory view	57
6.1	The three groups of tests and what each one checks	59
6.2	The off-board trace path and its tests	61
6.3	The live trace check driven against the device	65
6.4	The function trace view filled from a live capture	66
6.5	The instruction trace view filled from a live capture	67

List of Tables

1.1	Axes of debugging technique	4
1.2	Defect classes and the evidence available to investigate them	6
1.3	Trace capability of the STM32 families surveyed	8
1.4	Debug probe support across the proprietary environments	9
1.5	Positioning of this work against existing tools	10
1.6	Constraints on the work and their consequences	10
2.1	ARM Mapping Symbols	16
2.2	DWARF constructs and the operations they support	17
2.3	AMBA buses and their roles in debug and trace	18
2.4	Types of trace	21
2.5	Kinds of trace element	23
2.6	Trace Memory Controller configurations	25
2.7	Trace Memory Controller operating modes	26
2.8	On-chip and off-chip trace capture	26
3.1	Scope of the three deliverables	28
3.2	Functional requirements	29
3.3	Non-functional requirements	30
3.4	Traceability of requirements to deliverables	30
3.5	Trace components of the target device	32
3.6	Development environment	32
3.7	External software components	33
4.1	Provider categories	39
6.1	The four comparisons and their outcomes	68
6.2	Defects found on hardware	69
6.3	Requirements traced to components and their realization	69
6.4	Verification of the requirements	70
6.5	Measured figures for captured traces	70
6.6	Limitations of the trace pipeline	71

7.1	Registered protocol requests by area	76
7.2	Test inventory	77

List of Acronyms

- **ADI** = **ARM Debug Interface**
- **AHB** = **Advanced High-performance Bus**
- **AMBA** = **Advanced Microcontroller Bus Architecture**
- **APB** = **Advanced Peripheral Bus**
- **ARM** = **Advanced RISC Machine**
- **ATB** = **Advanced Trace Bus**
- **AXI** = **Advanced eXtensible Interface**
- **CMSIS** = **Cortex Microcontroller Software Interface Standard**
- **CTI** = **Cross-Trigger Interface**
- **DAP** = **Debug Access Port (ARM Debug Interface)**
- **DWARF** = **Debugging With Attributed Record Formats**
- **DWT** = **Data Watchpoint and Trace**
- **ELF** = **Executable and Linkable Format**
- **ETB** = **Embedded Trace Buffer**
- **ETF** = **Embedded Trace FIFO**
- **ETM** = **Embedded Trace Macrocell**
- **ETR** = **Embedded Trace Router**
- **FPB** = **Flash Patch and Breakpoint**
- **GCC** = **GNU Compiler Collection**

- **GDB** = **GNU Debugger**
- **IAR** = **Ingenjör**sfirm**a Anders Rundgren**
- **IDE** = **I**ntegrated **D**evelopment **E**nvironment
- **ITM** = **I**nstrumentation **T**race **M**acrocell
- **JTAG** = **J**oint **T**est **A**ction **G**roup
- **LLVM** = **L**ow **L**evel **V**irtual **M**achine
- **MCU** = **M**icro**c**ontroller **U**nit
- **MEM-AP** = **M**emory **A**ccess **P**ort
- **PPB** = **P**ri**v**ate **P**eripheral **B**us
- **RSP** = **R**emote **S**erial **P**rotocol
- **SB** = **S**cripting **B**ridge
- **SVD** = **S**ystem **V**iew **D**escription
- **SWD** = **S**erial **W**ire **D**eb**u**g
- **SWO** = **S**erial **W**ire **O**utput
- **TMC** = **T**race **M**emory **C**ontroller
- **TPIU** = **T**race **P**ort **I**nterface **U**nit

Introduction

Embedded systems operate under real-time constraints. They interact directly with hardware peripherals and with asynchronous external events. Both properties make them difficult to observe and to debug. Traditional techniques such as serial logging or software breakpoints are often insufficiently informative, and they are intrusive enough to perturb the behavior under examination. Modern ARM-based microcontrollers address these limitations by integrating hardware debug and trace capabilities through the ARM CoreSight architecture, which permits non-intrusive visibility into instruction execution, memory accesses and system activity.

STM32 microcontrollers expose a substantial subset of these capabilities. Despite the sophistication of the underlying hardware, the features remain locked behind proprietary tooling, or surface only as the limited workflows that commercial development environments choose to expose. Within mainstream open-source debugging environments, instruction trace is rarely treated as a first-class debugging primitive, and exists instead as a fragmented collection of low-level mechanisms requiring manual integration.

Using instruction trace in practice involves extracting raw trace data through custom scripts, decoding packets through a standalone framework such as OpenCSD[1], and relying on visualization or analysis tools disconnected from the debugging workflow itself. The resulting process lacks standardization, interoperability and accessibility. Advanced trace-assisted debugging is therefore placed outside the reach of many embedded developers, despite the hardware support already present on the devices they use.

This project addresses that gap through a study of the trace and debug infrastructure available on STM32 microcontrollers, spanning CoreSight components, transport protocols, debug probes, software tooling and host-side integration mechanisms. The work then proposes a practical and reproducible workflow for instruction trace capture, decoding and interpretation, designed to integrate with existing environments while remaining independent of proprietary abstractions.

The core of that workflow is a pipeline that turns a trace capture into the sequence of executed instructions expressed in source terms, obtained over the ordinary debug connection with the probe already present on the board. This pipeline is the main contribution of the project, and the chapters on architecture, implementation and validation return to it.

The report is organized in six chapters. The first presents the host organization and states the problem the project addresses, together with a survey of what existing tools provide. The second establishes the concepts the later chapters rely upon, covering debugging fundamentals, program representation, the ARM debug architecture and the CoreSight trace architecture. The third states the objectives, the requirements derived from them and the working environment. The fourth describes the architecture of the solution, its decomposition into components and the reasons behind each decision. The fifth describes the realization of the three deliverables. The sixth

reports the tests performed and the results obtained.

1 Context & Problem Statement

Introduction

This chapter states the setting of the project and the problem it addresses. It presents the host organization and the origin of the subject, establishes what an execution record supplies that the ordinary means of debugging do not, states the question the project answers, surveys what the devices provide and what the available tools expose of it, and closes with the objectives, the constraints and the deliverables.

1.1 Host Organization

This section presents the organization within which the project was carried out. It introduces STMicroelectronics, then the Tunis site that hosted the work, and then the team whose activity gave rise to the subject.

1.1.1 STMicroelectronics

STMicroelectronics (ST) is a global semiconductor manufacturer, founded in 1987 through the merger of SGS Microelettronica of Italy and Thomson Semiconducteurs of France. It supplies devices for the automotive, industrial, personal electronics and communications markets [2].

Among its product lines is the STM32 family of microcontrollers, built on the ARM Cortex-M processors. That family is the subject of this work, and Section 1.4 surveys its trace capability.

1.1.2 ST Tunis

The STMicroelectronics Tunis site, located in the El Ghazela Technopark, employs over 100 professionals in fields such as tools, applications, quality and support functions. The site contributes to circuit design, software tool development, application support, and the electrical validation of new circuits.

1.1.3 Application & Product Support Team

The Application & Product Support team provides technical support for ST's tools and products, and manages and publishes the STM32 product documentation. It answers customer questions on the use of the devices, maintains the accuracy of the documents that describe them, and produces application notes and training material

stating how a given capability is used in practice.

The subject of this project originates from concrete needs identified within the team and STMicroelectronics. In the course of its activities, the team routinely handles customer issues of varying complexity, constraints and contexts. While the majority of these problems can be resolved through conventional debugging means, the limited utilization of instruction trace capabilities has been identified as a potential factor slowing down the resolution and root-cause analysis of certain issues. Moreover, access to this feature is currently being tied to specific hardware and proprietary tools. This situation has given rise to a growing demand for an independent instruction trace implementation, free of hardware and vendor-specific dependencies, a request expressed by customers and one that remained as a backlog item since 2017. It is in response to this long-standing need that the present project was conceived.

1.2 Observing an Executing Program

A defect is a condition in a program that causes its behavior to violate its specification. To locate a defect, the developer must find the point at which the program departs from its specification, and finding that point requires observing how the program executed. Consider a program required to toggle a LED once per second. It may fail because a clock is not configured, or because it writes an incorrect register. These are two distinct defects that produce an identical symptom, so the symptom alone cannot separate them. Separating them requires observing the intermediate behavior of the program.

The techniques that supply this visibility are distinguished along three axes, and each technique occupies a position on every axis. Table 1.1 states the three axes that concern this work.

Table 1.1: Axes of debugging technique

Axis	Distinction
Intrusive and non-intrusive	Whether applying the technique alters the behavior of the program under examination.
Live and post-mortem	Whether the program is examined under the control of a debugger or from a record produced during execution and analyzed afterward.
Run-control and trace-based	Whether the developer directs the investigation by choosing where to stop, or reconstructs it from a recording of the program flow.

This section examines three ways of observing a program and places each on the three axes above. The first is halting the processor. The developer stops the program at chosen points and reads its state. This technique is live and run-control based, and it is intrusive, because stopping the processor alters the timing of

everything around it. The second is instrumenting the program, by adding code that makes the program report on itself as it runs. This technique is live and trace-based, and it too is intrusive, because the added code and the cost of transmitting its output alter how the program runs. The third is instruction trace. Hardware records the flow of the program as it executes, and the host reads that record once the program has stopped. This technique is trace-based and post-mortem, and it is non-intrusive, because the recording is done by dedicated hardware and adds no code to the program. The section then examines what each technique reveals, and establishes that one class of defect can be reached only by instruction trace.

1.2.1 State at a Halt

When the processor is halted, the host may read the register file, the contents of memory, the registers of the peripherals, and the registers of the debug components themselves. From the stack pointer and the debug information held in the program image, an unwinder reconstructs the frames that are still live.

Each of these is a view of the state the device holds at the instant of the halt. The state at a given instant describes the values the program holds at that instant. It carries no record of the path the program took to reach it. The order in which the instructions ran, the branches the program took, the calls that have already returned, and the handlers that have already completed are gone by the time the processor stops, and a later read cannot recover them. Where the stack itself has been damaged, the unwinder cannot proceed, and the live frames become unavailable as well.

Halting the processor is also less passive than it appears. The processor stops, but the rest of the system keeps running. Timers continue to count, transfers already programmed into a direct memory access controller continue to run, and a peer on a communication link observes a device that has stopped responding. Some devices provide controls that freeze selected peripherals while the processor is halted. Whatever lies outside the device cannot be frozen by these controls.

1.2.2 The Cost of Instrumentation

The alternative to halting is to instrument the program, so that it reports on itself as it runs. Transmitting that report has a cost, and that cost falls on the system under examination.

Consider a character sent over a UART configured for 115 200 baud, with eight data bits, no parity, and one stop bit. That character occupies ten bit periods, which is 86.8 μ s on the wire. On a processor clocked at 600 MHz, those 86.8 μ s are roughly 52 000 cycles for a single character, and a blocking implementation spends all of them waiting.

Moving the transmission to a direct memory access controller removes the stall, but it introduces a perturbation of its own. The controller becomes a second master on the bus matrix, and it contends for the same memory banks and the same slave ports as the instruction and data ports of the processor. It also raises

an interrupt on completion. A timing-dependent defect is most sensitive to two factors, the arbitration on the interconnect and the interrupt load of the system, and instrumenting the program alters both.

1.2.3 Defects Requiring Execution History

Some defects lie beyond the reach of both techniques. Table 1.2 states the principal cases. For each case, it gives the evidence available once the processor has stopped, and the evidence that a record of the execution supplies.

Table 1.2: Defect classes and the evidence available to investigate them

Defect	Available once halted	Supplied by an execution record
Memory corrupted by a stray write	The corrupted contents, and the address at which the program eventually faults	The instructions that ran before the corruption, among them the store that performed it
Fault taken with a damaged stack	The fault status registers, and the stacked frame where it survives	The calls executed before the fault, recovered from the record when the stack can no longer supply them
Race between a handler and the main program	Inconsistent state, observed after both have run	The order in which the two ran, and the points at which preemption occurred
Fault occurring before a debugger attaches	Nothing, since the device has already failed	The execution from the reset handler onward
Defect appearing rarely, or only at full speed	Only what a reproduction under the debugger yields, if it reproduces at all	A record of the single occurrence, obtained without altering the timing of the program

All of these cases share a single requirement. To investigate them, the developer must know the order in which the instructions executed, alongside the state the program held at the halt. They therefore call for a technique that is non-intrusive, that permits post-mortem examination, and that is trace-based. Of the three techniques, only instruction trace holds all three of these positions.

The same record serves purposes beyond locating defects. Because it records which instructions executed and how often, it yields code coverage without instrumenting the program, and it identifies the paths on which execution time is spent.

1.3 Problem Statement

Such a record is produced by hardware. Much of the STM32 range carries a trace unit that reports the flow of the program as it executes, and part of the range carries a buffer into which that report is captured without leaving the device. The hardware alone does not make the record usable. It must be configured, the capture must be retrieved, and the result must be interpreted against the program image before it means anything to a developer. Each of those steps requires software, and outside the environment of a tool vendor that software does not exist.

The question this project answers is the following. How are the instruction trace capabilities of STM32 devices made reachable within an open and integrated debug chain, without a dedicated trace probe and without dependence on a proprietary environment?

The question divides into four. Configuration asks how a host configures the trace components of a device over the ordinary debug connection, when their presence, their placement and their addresses are properties of the particular device, which the architecture leaves unspecified. Capture asks how the record is retained and retrieved when no trace pins and no capture hardware are available. Interpretation asks how a capture is turned back into the instructions that executed, and whether that is done independently of the session that produced it. Restitution asks how the result is presented in source terms, inside a development environment the developer already uses. The chapters that follow answer them in turn.

1.4 State of the Art: Instruction Trace on ARM Microcontrollers

The survey below is restricted to instruction trace on ARM M-profile microcontrollers. Conventional debugging on these devices is not surveyed, since breakpoints, memory inspection and single stepping are supported uniformly by every tool considered here and raise no question this project needed to answer. Three bodies of work bear on instruction trace specifically: what the devices provide, what the available tools expose of it, and what published work has attempted with it.

1.4.1 Trace Capability of the STM32 Devices

Whether instruction trace is available at all is a property of the device. A family implements a trace unit or it does not, and independently of that it carries a buffer able to hold the captured trace inside the device or it does not. Where no such buffer exists the trace leaves only through dedicated pins, which requires those pins to be routed on the board and hardware able to capture them.

Table 1.3 states the capability of the families surveyed. It follows AN4989 [3] where that document covers the family, and the reference manual of the family otherwise, of which RM0433 [4] and RM0477 [5] describe the two devices carrying an on-chip buffer that this work uses.

Table 1.3: Trace capability of the STM32 families surveyed

Family	Processor	Instruction trace	Trace unit architecture	On-chip trace
C0, F0, G0, L0, U0	Cortex-M0 and M0+	✗	–	✗
F1, F2	Cortex-M3	✓	ETMv3	✗
F3, F4, G4, L4, L4+	Cortex-M4	✓	ETMv3	✗
F7	Cortex-M7	✓	ETMv4	✗
H7, H7RS	Cortex-M7	✓	ETMv4	✓
C5, H5, L5, U3, U5	Cortex-M33	✓	ETMv4	✗
N6	Cortex-M55	✓	ETMv4	✓

The last two columns are read together. Every family carrying an on-chip buffer implements the later trace unit architecture, so an implementation supporting that architecture alone loses no device reachable without trace pins. The converse does not hold, since several families implement it and carry no buffer, and on those the trace leaves through the pins. The capability the hardware offers is therefore not the question. What the available software does with it is.

1.4.2 Proprietary Development Environments

Instruction trace on ARM microcontrollers is exposed principally by the development environments of IAR, Keil and SEGGER. Each couples a debugger to a debug probe of its own make, and reaching the trace means working inside that environment with that probe [6].

These environments expose a fixed set of tracing settings through their interface, and reapply that configuration at every resume. The behavior is consistent from the point of view of the tool, and it leaves a developer no means of holding a configuration of their own.

The probes impose limitations of their own. IAR, Keil and SEGGER each supply a base model and an extended model, the latter capturing trace through dedicated pins. Table 1.4 states which environment accepts which probe.

Among the environments examined, none exposes capture from the on-chip buffer through a probe other than its own, and none decodes a capture outside a live debug session or a capture obtained by means other than a debug probe. The probes fitted to development boards, namely ST-LINK and CMSIS-DAP, are accepted by every environment and capture no trace through pins, so the developer who owns no other probe is left with no route to the trace.

Table 1.4: Debug probe support across the proprietary environments

Probe	Supported by IAR	Supported by Keil	Supported by SEGGER	Trace pin capture
I-jet	✓	✗	✗	✓ ¹
ULINK	✗	✓	✗	✓ ¹
J-Link, J-Trace	✓	✓	✓	✓ ¹
ST-LINK	✓	✓	✓	✗
CMSIS-DAP	✓	✓	✓	✗

¹ Given the appropriate model

1.4.3 Open-Source Debug Software

The open-source tools divide the problem differently and stop at a different point in it.

OpenOCD [7] is the debug server in general use for ARM microcontrollers, and it reaches the debug port, the access ports and the memory of the target. Its support for trace units covers the architecture implemented in the ARM9 generation of processors, and its manual describes that support as little used and possibly unreliable. It offers no driver for the on-chip buffer of the devices considered here.

pyOCD [8] reaches the debug port over the same class of probe and exposes instrumentation trace only. Instrumentation trace answers a different question. It reports what the program was written to announce, which is distinct from the instructions the processor executed.

The decoder itself exists. OpenCSD [1] implements the trace protocols of the current architecture and is the decoder used by the Linux kernel profiling infrastructure to interpret captures taken on application-profile processors [9]. That arrangement presumes a host running on the traced device, kernel drivers configuring the trace components from within it, and a processor of the application profile. On a bare-metal microcontroller the host is external and reaches the device only through the debug port, and none of the surrounding infrastructure transfers. The decoder is available and what stands on either side of it is not.

1.4.4 Published Work

Instruction trace has been applied in published work to problems adjacent to debugging. FrankenTrace [10] concerns execution tracing on Cortex-M systems-on-chip, and Ninja [11] concerns tracing and debugging on ARM processors. Both establish that the hardware supports uses beyond the ones the vendor environments expose.

1.4.5 Positioning of this Work

Table 1.5 sets the approach of this work against the existing tools, across the capabilities that bear on instruction trace.

Two intersections are occupied by nothing. The first is capture from the on-chip buffer through a

Table 1.5: Positioning of this work against existing tools

Capability	Proprietary environments	Open-source tools	This work
Conventional debugging	✓	✓	✓
Instruction trace	✓	✗	✓
On-chip trace capture	✓ ¹	✗	✓
No dedicated trace probe	✗	–	✓
Offline decoding	✗	✓ ²	✓
IDE integration	✓	✓	✓
Open source	✗	✓	✓

¹ Reachable only through the vendor’s own probe.

² A decoder operates on a capture independently of a session, and nothing supplies it with one.

probe the developer already owns, which the proprietary environments withhold and the open tools have not implemented. The second is interpretation decoupled from the session that produced the capture, for which a decoder exists and no path to it does. This work occupies both.

1.5 Objectives, Constraints and Deliverables

The objective follows from the question of Section 1.3. The trace hardware is configured over the ordinary debug connection, the capture is retrieved from the on-chip buffer and reconstructed against the program image, and the executed instructions are presented in source terms inside a development tool during a live debug session, the reconstruction remaining usable on a capture taken independently of any session.

1.5.1 Constraints

The work is bound by the conditions of its setting. Table 1.6 states each constraint, its origin and what it excludes.

Table 1.6: Constraints on the work and their consequences

Constraint	Origin	Consequence
No hardware beyond the probe fitted to the development board	The result is reproducible by any owner of the board, without further purchase	Capture through trace pins is excluded, and the on-chip buffer is the only route
No dedicated trace probe	The probes that capture trace are proprietary to one environment each	The capture is retrieved over the ordinary debug connection
Windows and Linux	The teams and the customers concerned use both	No dependence on facilities specific to one platform
Integration with existing tools	A separate tool would not be adopted alongside an existing workflow	The result plugs into a development environment already in use

The first constraint carries a consequence worth stating here. A buffer inside the device holds a bounded quantity of trace. A capture is therefore a window of the most recent execution, and its extent is fixed by the memory the device can spare. Section 6.8 states the quantity of trace measured for a capture taken on the target.

1.5.2 Deliverables

The work is delivered in three parts. An application note gathers into one document the procedure for using instruction trace on these devices. A proof of concept demonstrates that a capture is obtained from an on-chip sink, decoded against the program image, and reconstructed into the executed instructions. A debugger carries that capability into a development tool, alongside breakpoints, memory and variable inspection, and disassembly.

Two of these rest on components adopted from existing projects. The debug server and the debug engine are existing open-source projects, and the decoding of the trace protocol is performed by an existing library. Written for this work are the drivers that configure the trace components and retrieve the capture, the chain that reconstructs the executed instructions from it, the components that constitute the core of the debug engine, and the extension that presents the result. Chapter 5 states that division component by component.

1.6 Conclusion

This chapter has established that the ordering of a program's execution is required to locate an important class of defect, and that neither halting the processor nor instrumenting the program supplies it. It stated the question the project answers, namely how the instruction trace capability of STM32 devices is made reachable in an open and integrated debug chain without a dedicated trace probe, and divided that question into configuration, capture, interpretation and restitution. The survey established that the hardware provides the capability, that the proprietary environments expose it only through their own probes, and that the open tools reach a decoder and no capture. The objective, the constraints and the three deliverables were stated. The chapter that follows establishes the concepts on which the answer rests.

2 Foundations of Debugging and Instruction Trace

Introduction

This chapter establishes the concepts on which the remainder of this report depends. The presentation moves from the general to the particular. It opens with the definition of debugging and of the tool that performs it, then describes the operations a debugger provides and the categories into which debugging techniques fall. It continues with the representation of a compiled program, then narrows to microcontrollers, to the ARM M-profile architecture, to the debug and trace infrastructure that ARM defines for it, and finally to the implementation of that infrastructure in STM32 devices.

2.1 Debugging Fundamentals

This section states what a debugger is and how it communicates. It describes the components a debugger is built from, and then the two protocols that connect those components, namely the Debug Adapter Protocol toward the frontend and the GDB Remote Serial Protocol toward the target.

2.1.1 Anatomy of a Debugger

A debugger is a software system that provides controlled interactions with a target system generally referred to as a *debuggee* for the purpose of examining and modifying its state.

Providing these operations requires two broad classes of work. The first is interpreting the representation of the program, which covers the machine code and the debug information held in the executable image. The second is interacting with the target.

A further distinction concerns presentation. The *frontend* is the program that provides the user interface through which debug operations are requested and the state of the debuggee is displayed. The *backend* is the part that carries out the two classes of work mentioned above and maintains the state of a debugging session. For embedded systems such as a microcontroller, a hardware component named *debug probe* or *hardware probe* is generally needed to connect the host machine and the target debuggee. This element represents an intermediary

communication channel between both systems. Figure 2.1 shows the decomposition.

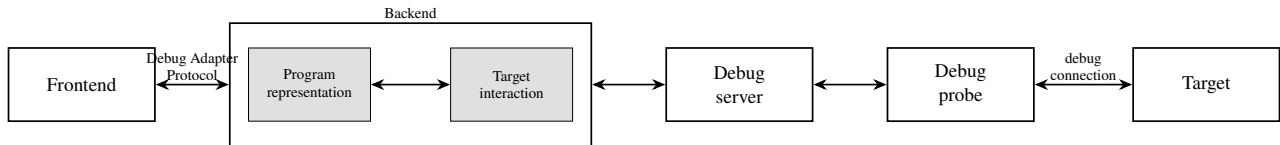


Figure 2.1: Conceptual decomposition of a debugger

Each junction in this chain is governed by a protocol. The connection between the probe and the target depends on the architecture of the device and is the subject of Section 2.6. The decomposition given here is conceptual, and a given system may distribute the responsibilities differently.

2.1.2 The Debug Adapter Protocol

The Debug Adapter Protocol defines the messages exchanged between a development tool and a *debug adapter*, a component that translates those messages into operations on the debugging mechanism beneath it [12]. A debugger is therefore not required to implement the protocol itself, since the adapter stands between the protocol and the interface the debugger already exposes [12].

The protocol describes the entities the developer works with, namely source files, line numbers, variables, threads and stack frames. It further permits requests and events outside the standard set, which allows an adapter to expose capabilities that the protocol does not describe.

2.1.3 The GDB Remote Serial Protocol

The GDB RSP is a point-to-point text-based communication protocol defined as part of the GNU debugger [13], that allows a debugger to connect to a remote target.

The debugger is the client and the target is the server. Every exchange is initiated by the client, and every transaction must be completed before initiating the next one[13].

The operations the protocol carries include reading and writing the processor registers, reading and writing memory, inserting and removing breakpoints, resuming execution and executing a single instruction [13].

A debugger may extend the set of packets. LLDB for example, defines packets of its own alongside the standard set [14].

2.2 Debug Primitives

In this work, *debug primitives* refer to the fundamental operations that a debugger exposes. Each is defined here in general terms.

2.2.1 Run Control

Run control provides mechanisms for starting, stopping and controlling the execution of the debuggee. Halting suspends execution and leaves the state of the program unchanged. Resuming returns the program to normal execution from the point at which it was suspended.

Stepping executes a limited portion of the program and then halts again. The size of that portion depends on the granularity requested, mainly, a single instruction or one source statement, which usually amounts to several machine instructions [15]. Further granularities are common, such as advancing over a complete function call without entering it.

2.2.2 Breakpoints: Software and Hardware

A breakpoint is an artificial stopping point inserted into a program for the purpose of debugging. When execution reaches the breakpoint, the program is suspended, after which execution may continue from the point at which it was suspended [15].

The mechanisms by which breakpoints are implemented are divided into two categories:

A *software breakpoint* replaces the instruction at the target address with a dedicated instruction that raises a debug event, and restores the original instruction once the breakpoint is removed.

A *hardware breakpoint* instead configures dedicated hardware components on the device such as comparators that watch the address of instructions fetched and raises a debug event on a match without modifying the program. Figure 2.2 contrasts the two mechanisms.

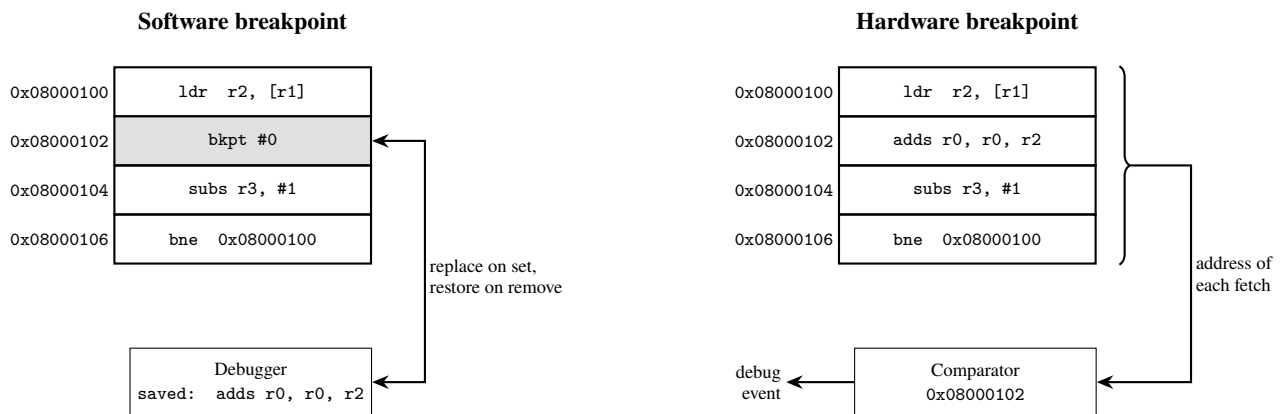


Figure 2.2: Software and hardware breakpoint mechanisms

2.2.3 State Inspection: Registers, Memory and Variables

The state of a program is observed through two elementary operations, which are reading the processor registers and reading memory [15].

These two operations are independent of the program, as neither carries the name, the type or the extent

of the variable present in the source code. Presenting a variable therefore requires three further pieces of information, and is generally supplied by the debug information described in Section 2.3.3.

2.2.4 Call Stacks and Unwinding

The call stack is the sequence of subroutine activations that are in progress. An activation consists of a code location within the subroutine, an area of memory allocated on the stack termed a call frame, and the set of registers in use by the subroutine at that location [16].

Only the innermost activation is directly visible, since the processor registers describe that activation alone. Reaching the others requires *unwinding*, which proceeds one frame at a time. Starting from the current activation, the debugger restores the registers that this activation preserved and computes the address of the preceding call frame together with the corresponding code location[16].

2.2.5 Disassembly

Disassembly is the translation of the bytes held in the executable memory of the target back into machine instructions.

Disassembly serves two purposes in a debugger. It allows a developer to inspect code for which no source is available, and it allows the machine instructions produced by the compiler to be compared against the source statement they were generated from.

2.3 Program Representation

This section describes how a compiled program is stored, what the mapping between source-code and machine-code contains, and how machine code is translated back into instructions.

2.3.1 The Embedded Build Model

A program written in a high-level language passes through several tools before it can execute. The compiler translates each source file into an object file containing machine code together with unresolved references to entities defined elsewhere. The linker combines the object files, resolves those references, assigns an address to every function and variable, producing an executable image.

Embedded development has different properties in this aspect to host machines, mainly the fact that addresses are fixed at link time and describe a physical device.

The debugger consumes the same image. extracting machine code to be disassembled, the addresses at which functions and variables reside, and the debug information relating those addresses to the source.

2.3.2 The ELF Format

The Executable and Linkable Format (ELF) is the object file format used by the toolchains considered in this work. ARM defines a processor-specific supplement to it for its own architecture [17].

2.3.2.1 Segments, Sections and the Loadable Image

An ELF file describes its contents in two manners. The section header table describes the file as a set of *sections*, which groups related content under a name and a type, so that machine code, initialized data, uninitialized data and debug information each occupy sections of their own. The program header table describes the file as a set of *segments*, which is the view used when the program is loaded. A segment of loadable type identifies a range of the file together with the address at which that range must appear in memory.

2.3.2.2 Symbol Tables

A symbol table is a data structure where each symbol in the source code is associated with the necessary information for its functioning. Each entry records the name of an entity, its address, the number of bytes it occupies, and its kind, which distinguishes a function from a data object among other categories.

2.3.2.3 ARM Mapping and Tagging Symbols

A section of an ARM ELF file can contain a mixture of ARM code, Thumb code and data. The mapping and tagging symbols are used in the symbols table to denote the start of a specific section of the appropriate type.

Table 2.1: ARM Mapping Symbols

Symbol	Definition
\$a	Denotes the start of a region of code containing ARM instructions.
\$t	Denotes the start of a region of code containing Thumb instructions.
\$d	Denotes the start of a region of data.

2.3.3 DWARF Debug Information

DWARF is the debug format used by the toolchains considered here. It describes a program as a tree in which nodes represent functions, data and types, provides a mapping between executable instructions and the corresponding source code lines, and describes how to unwind the stack [15]. It is stored in dedicated sections of the ELF file. Table 2.2 states the constructs it defines and the debugger operation each supports.

Table 2.2: DWARF constructs and the operations they support

Construct	What it describes	Operation it supports
Debugging information entries	The program as a tree of functions, data and types, nested as the source is nested.	Determining which names are visible at a given point.
Line number program	The correspondence between instruction addresses and source lines.	Presenting a location in source terms and stepping from line to line.
Inlined subroutines	The expansion of a subroutine at its call site, and the chain of source locations that results.	Recovering the call site of inlined code.
Type descriptions	How the bytes belonging to a variable are to be interpreted.	Displaying a typed variable from its bytes.
Location descriptions	Where an object resides, which may change over its lifetime.	Finding a variable at the current point of execution.
Call frame information	Where registers are saved and how the address of the preceding frame is computed.	Unwinding the call stack.

2.4 Microcontrollers

A microcontroller is a system that integrates a processor, its memories and a set of peripherals within a single component. It cannot host a debugger, since it has neither the memory to hold one nor the means for a developer to operate one on it [15].

2.4.1 Device Description

The CMSIS System View Description format formalizes the description of the system contained in ARM Cortex processor-based devices, in particular, the memory mapped registers of peripherals. The detail contained in system view descriptions is comparable to the data in device reference manuals. The information ranges from high level functional descriptions of a peripheral all the way down to the definition and purpose of an individual bit field in a memory mapped register. [18].

The format is based on XML and describes a single device per file. A device consists of a processor and at least one peripheral, each peripheral contains at least one register, and a register may consist of one or more fields [18]. Each level carries attributes, such as the address of a register, its reset value, the width and position of each field, whether a field may be read or written, and the meaning of particular field values.[18].

2.5 The ARM Cortex-M Architecture

This section describes the parts of the ARM Cortex-M architecture on which the debug and trace infrastructure depends. ARM divides its architecture into three profiles addressed to different classes of application. The A-profile targets systems running general-purpose operating systems, the R-profile targets systems with real-time requirements, and the M-profile targets microcontrollers, which is the profile this work concerns.

2.5.1 AMBA Buses and their Roles in Debug

The components of an ARM device are connected by buses defined in the Advanced Microcontroller Bus Architecture (AMBA). Table 2.3 states the four most common ones.

Table 2.3: AMBA buses and their roles in debug and trace

Bus	Name	Role
APB	Advanced Peripheral Bus	Carries configuration accesses to debug components. Transactions arriving from a debug connection are converted into APB transactions and transported to each of the components the debugger must access [19].
AHB	Advanced High-performance Bus	The protocol through which the debug registers of a Cortex-M processor are commonly reached. Accesses may be directed to every location in the memory space of the processor, the debug registers among them [19].
AXI	Advanced eXtensible Interface	Carries accesses into the main functional memory space of the devices that implement it, bypassing the caches and the memory map controls of the processor [19]. Where a device routes captured trace into system memory, this is also the interface over which that trace is written, as Section 2.7.6 describes.
ATB	Advanced Trace Bus	Reserved for trace. It is a common bus used by the trace components to pass trace data in a data-agnostic format [20], which is what allows one infrastructure to transport several kinds of trace.

2.6 The ARM Debug Architecture

Section 2.1.1 left the junction between the probe and the target undescribed. This section describes the specification that governs it and the debug logic it reaches.

2.6.1 The ARM Debug Interface

The ARM Debug Interface (ADI) defines the means by which a debugger accesses the debug functionality provided by debug components in an embedded system on chip [21].

An implementation of that specification within a device is termed a Debug Access Port (DAP), which provides a debugger with a standard interface for accessing debug resources in systems that expose their debug information by resource-specific methods [21].

The Debug Access Port presents one interface to the debugger and performs the resource-specific access on its behalf. It is composed of two kinds of elements, a Debug Port and one or more Access Ports, Figure 2.3 shows an example configuration of a DAP.

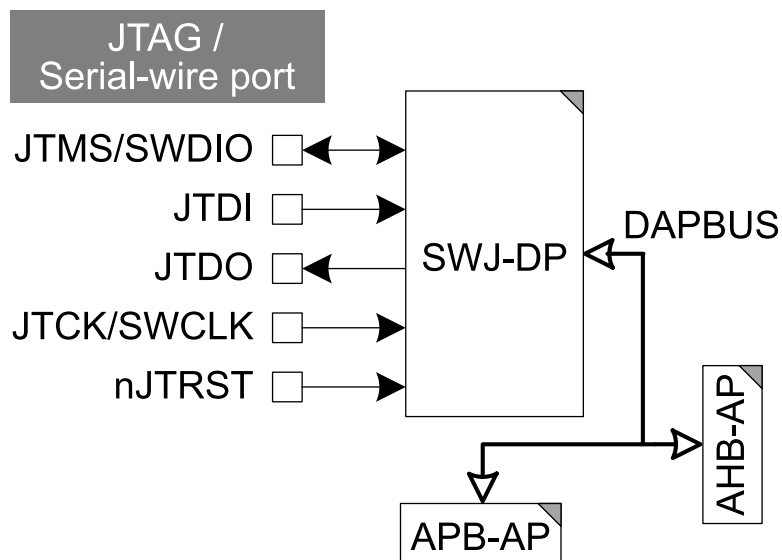


Figure 2.3: Structure of a Debug Access Port[5]

2.6.2 Debug Port: JTAG and SWD

A Debug Port provides the external physical connection to the interface and the methods by which the debugger accesses the Debug Port and Access Port registers [21].

There are three types of Debug Ports,

- JTAG Debug Port is accessed by scan chains compliant with IEEE 1149.1.
- Serial Wire Debug Port (SWD) is a two-pin serial interface using a packet-based protocol to read and write registers.

- SWJ Debug Port which combines the two above[21].

The debugger drives serial accesses that scan values into registers inside the Debug Port, which are then converted into bus transactions directed at an Access Port register [19].

2.6.3 Access Ports and the Memory Access Port

An Access Port uses a resource-specific transport mechanism to access debug information in the system, and passes that information to the Debug Port [21]. A Debug Access Port contains at least one Access Port and may contain several [21], each driving a different memory space, with the type of Access Port determined by the bus protocol required for that space [19].

2.6.4 Component Identification

CoreSight allows a debugger to discover the debugging capabilities of an unfamiliar device through a common register layout. A component compliant with the CoreSight architecture occupies a block with a size that is a multiple of four kilobytes, divided between a set of predefined management registers and a set of registers specific to the device [19].

2.6.5 Reset Types and Vector Catch

M-profile architectures define two types of reset, A Local reset is one initiated through the Debug Access Port, and resets only the core and the debug logic, a System reset on the other hand resets the entire SoC. Vector Catch is a debug mechanism implemented in Cortex-M devices that allows the device to enter Debug State upon exception entry without requiring a breakpoint.

2.6.6 Breakpoint and Watchpoint Units

The Flash Patch and Breakpoint unit (FPB) is the debug component that implements hardware breakpoints. Its relevant capability is that it can set breakpoints on code held in read-only memory [22].

The Data Watchpoint and Trace unit (DWT) implements generic comparators that can be configured to provide various functionalities such as watchpoints, data tracing or signaling for external resources. Moreover, these comparators can be configured to signal a match on either an instruction address, a data address or a data value given that the prior comparator is a data address one.

2.7 The CoreSight Trace Architecture

This section describes the architecture that ARM defines for trace. CoreSight is the mechanism ARM uses to provide the connectivity between a debugger and a processor, and it defines a set of capabilities that may be

designed into a processor or into system-level components [19].

2.7.1 Trace Sources, Links and Sinks

Trace components fall into three categories. A trace source is a component that generates trace data. A trace sink is a component that stores or outputs trace data. A trace link is a component that connects trace components together [23]. Any trace path in a device is therefore a chain that begins at a source, passes through zero or more links, and terminates at a sink.

Since a design is free to implement any subset of these components, the topology differs from device to device. A debugger consequently needs to know the topology of the particular target before it can configure a trace path [23].

2.7.2 Instruction, Data and Instrumentation Trace

Trace refers to the process of capturing data that illustrates how the components in a design are operating, executing and performing, and the ARM approach usually involves a separate generation component for each type of trace [23]. Table 2.4 states the types that concern this work.

Table 2.4: Types of trace

Type	What it records	Intrusive
Instruction	The execution history of the processor, and the places where execution behaves unexpectedly [23]. This is the type on which this project is based.	X
Data	Each data access, together with the address and the value transferred [23]. The trace architecture leaves it optional [23].	X
Instrumentation	Events and values supplied by the software itself, in the style of a print statement [23].	✓

Instrumentation trace differs from the other two in that the program must be modified to produce it, which places it on the intrusive side of the distinction, albeit at a low level of intrusion [22].

2.7.3 Trace Source: the Embedded Trace Macrocell

The component that generates instruction trace is termed a trace unit. It traces the instructions a processor executes and the data those instructions act upon [24]. The implementation ARM supplies for this purpose is the Embedded Trace Macrocell (ETM), which exists in four architecture versions [24], [25]. This work treats ETMv4.

2.7.3.1 Functions of a Trace Unit

A trace unit has two main functions. The first is to generate trace, by producing trace streams. The second is to filter those streams, so that tracing may be made active only for particular threads of execution or particular functions [24].

The second function follows from a property of the first. A processor executes on the order of hundreds of millions of instructions per second, whereas the path that carries the trace has a fixed bandwidth and the storage at its end a fixed size. Filtering lowers the rate at which trace is produced, so that the path and the storage can keep up.

2.7.3.2 Trace Streams and Trace Identifiers

A trace unit conforming to ETMv4 generates an instruction trace element stream and, where data tracing is implemented and enabled, a data trace element stream [24]. The two are independent, and the architecture requires neither that they be captured by the same device nor that their capture be ordered [24]. Interpreting the data stream, however, typically requires the instruction stream to be interpreted as well [24].

Several streams may share one bus. Each stream therefore carries its own identifier, and Section 2.7.4 describes how it travels with the data.

2.7.3.3 Trace Elements

A trace unit reports execution as a stream of elements [24].

The stream does not contain an element for every executed instruction. An element is generated when the processor executes a direct branch, an indirect branch or an instruction synchronization barrier, when it takes or returns from an exception, and when it enters Debug state [24]. A wait-for-interrupt or wait-for-event instruction generates one only where the unit reports that it does [24]. Each element therefore implies the execution of every instruction from the target of the previous element up to the instruction it reports [24]. A conditional branch needs one bit, namely whether it was taken, and the instructions between two branches need none, since the program image already holds them. An *atom* element carries that bit.

Addresses follow the same principle. A branch whose target is encoded in the instruction needs no address in the trace, since the target may be read from the program image. The trace unit therefore omits an address wherever it can be inferred from the program image and from what the stream has already reported, and transmits only those that cannot [24].

Table 2.5 states what each kind of element reports.

Table 2.5: Kinds of trace element

Element	What it reports
Atom	Whether a conditional branch was taken.
Address	A branch target that cannot be inferred from the program image.
Exception and exception return	Entry to an exception and return from it, which occur at an instruction boundary with no branch appearing in the code.
Synchronization	Where a packet starts, the configuration of the trace run, and the address from which to begin reading.
Overflow	A gap in the trace, and the reason for it.
Timestamp and cycle count	Values from a global counter and counts of processor clock cycles.

Synchronization elements are generated periodically, which matters because trace held in a circular buffer is overwritten from an arbitrary point. A captured trace is therefore not necessarily continuous, and an analysis must represent a gap explicitly, keeping the two sides of it distinct [24]. Figure 2.4 works the principle through for a loop of four iterations, in which three taken atoms and one not-taken atom stand for sixteen executed instructions.

Program	Trace elements	Reconstructed execution
0x08000100 ldr.w r2, [r1], #4	Address 0x08000100	E { 0x08000100 ldr.w r2, [r1], #4 0x08000104 add r0, r2 0x08000106 subs r3, #1 0x08000108 bne.n 0x08000100
0x08000104 add r0, r2	Atom E	
0x08000106 subs r3, #1	Atom E	
0x08000108 bne.n 0x08000100	Atom E	
	Atom N	E { 0x08000100 ldr.w r2, [r1], #4 0x08000104 add r0, r2 0x08000106 subs r3, #1 0x08000108 bne.n 0x08000100
	<i>E: branch taken N: not taken</i>	
		E { 0x08000100 ldr.w r2, [r1], #4 0x08000104 add r0, r2 0x08000106 subs r3, #1 0x08000108 bne.n 0x08000100
		N { 0x08000100 ldr.w r2, [r1], #4 0x08000104 add r0, r2 0x08000106 subs r3, #1 0x08000108 bne.n 0x08000100

Figure 2.4: Trace elements generated for a short loop. Synchronization elements are omitted.

2.7.3.4 Packet Encoding

Elements are encoded into packets for transmission. A packet is a header byte followed by zero or more payload bytes, and the packet type determines how many [24]. A header is unique within a stream, though the same byte

may mean one thing in the instruction stream and another in the data stream, so a decoder must know which stream it reads [24].

Elements and packets do not correspond one to one. One atom packet may carry several atom elements, and six atom packet formats exist for that purpose [24]. Decoding is therefore two operations, namely recovering packets from the byte stream and expanding them into elements.

The trace alone does not state which instructions executed, since it reports branches and omits the addresses it expects to be inferred. Reconstruction requires the program image [24]. The image must be the exact one the processor ran, as any difference between it and the executed code corrupts the decoded output [23].

2.7.4 The CoreSight Trace Formatter

Where several trace streams share one path, the identity of each must travel with its data. Formatters are the method by which trace source identifiers are wrapped into the output trace data stream, and the format specifies how trace sinks embed the source identifiers carried on the ATB into a single stream [26].

The protocol outputs data in frames of sixteen bytes, arranged as Figure 2.5 shows. The architecture specification gives the meaning of every byte in full [26].

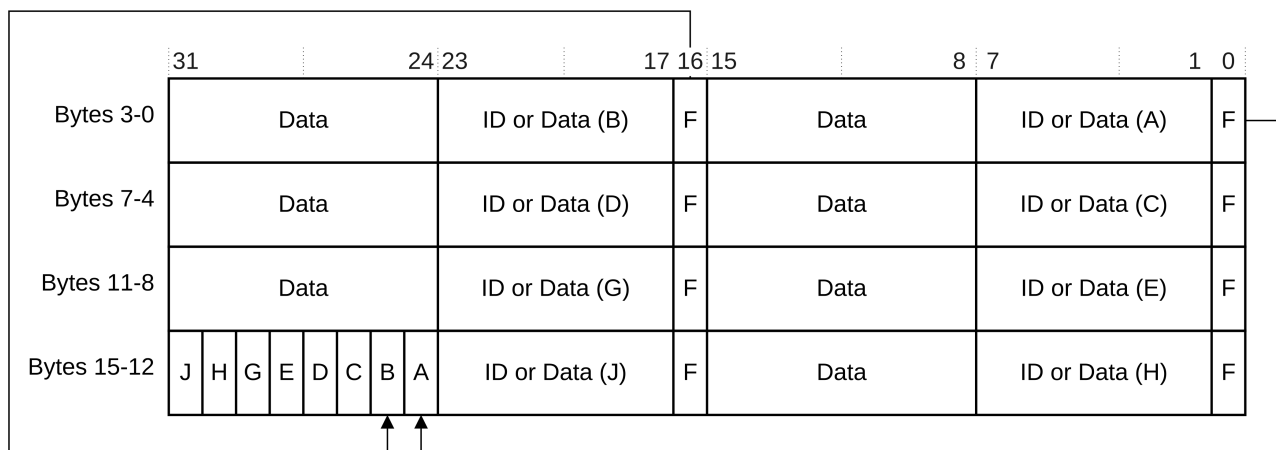


Figure 2.5: Structure of a formatter frame [26]

The design serves several ends at once [26]:

- Trace from several sources may be merged into a single stream and later separated.
- No requirement is placed on the data emitted by the sources.
- The result may be stored as a bitstream without separate alignment information.
- The result may be decoded even where the start of the trace has been lost.

Where only a single trace source is used and the embedded trigger and flow control information is not required, formatting may be disabled to obtain better throughput [26]. A decoder must therefore be told whether the stream it receives is formatted, since a formatted stream requires the frames to be unwrapped before the packet decoding of Section 2.7.3.4 may begin.

2.7.5 Off-Chip Trace Sinks: TPIU and SWO

Trace sinks fall into two families, distinguished by where the trace data comes to rest. The first exports it from the device. The Trace Port Interface Unit (TPIU) drives the external pins of a trace port so that capture hardware may receive the data [27], and the Serial Wire Output (SWO) passes trace over a single pin to a debugger at a lower rate [23]. This route is bounded by the bandwidth of a few pins running below the internal frequency, and it requires pins available on the package together with a probe able to capture them, which not every board provides [19].

2.7.6 On-Chip Trace Sinks: the Trace Memory Controller

The second stores it inside the device, which is what the Trace Memory Controller (TMC) does [23].

Two axes describe the component. The configuration is fixed when the design is integrated into a system, and Table 2.6 states the three that exist. The operating mode is chosen when the component is programmed, and Table 2.7 states the three that exist [28].

Table 2.6: Trace Memory Controller configurations

Configuration	Storage	Retrieval	Memory size range
ETB	Dedicated SRAM	Debug bus	512 bytes to four gigabytes
ETF	Dedicated SRAM	Debug bus, or drained over the trace bus	512 bytes to four gigabytes
ETR	System memory, over AXI	Debug bus	The size of the memory region

Table 2.7: Trace Memory Controller operating modes

Mode	Description	Configurations
Circular buffer	The storage is used as a circular buffer. Old trace is overwritten once it is full, and nothing may be read until capture has stopped.	ETB, ETF, ETR
Hardware FIFO	The component acts as a link between a source and a sink, applying back pressure to preserve trace.	ETF
Software FIFO	The component acts as a queue that a debugger reads over the debug bus while capture continues.	ETB, ETF, ETR

The dedicated memory of the ETB and ETF configurations is normally small, so triggering and filtering are usually necessary to ensure that important trace is not lost through wrapping [23].

Table 2.8 sets the two routes against each other.

Table 2.8: On-chip and off-chip trace capture

	On-chip	Off-chip
Bandwidth	The full internal bus rate	A fraction of the internal rate
Size	The size of the memory	The size of the capture device
Throughput	Read out after capture	Continuous during capture
Extra hardware	None	Trace pins, board routing and a capture probe

2.8 Conclusion

This chapter has assembled the concepts on which the remainder of the report rests. It defined a defect in terms of a specification, and a debugger as the system that examines and controls a program in order to locate one. It described the operations such a system provides and the protocols that connect its parts. It then set out the representation of a compiled program, comprising the object file that holds the image, the debug information that relates that image to the source, and the decoding that recovers instructions from the bytes. It narrowed to microcontrollers and to the ARM M-profile, described the ADI through which a debugger reaches a target, and described the CoreSight architecture through which a device reports what its processor executed.

Two results of this chapter are used repeatedly in what follows, and they belong to the two halves of it. The trace infrastructure of Section 2.7 produces a record of execution expressed in addresses, and produces it in a form that cannot be interpreted without the program image. The debug information of Section 2.3 converts an address into a function, a source file and a line. Neither is sufficient alone. Combining them is what turns a

buffer of trace bytes into an account of a program in the terms in which it was written, and Chapters 4 and 5 describe how this combination was achieved.

3 Requirements Specification and Work Environment

Introduction

Chapter 1 stated the problem the project addresses, together with its objectives and deliverables. This chapter specifies the scope of those deliverables, the requirements the work must satisfy, and the hardware and software environment in which it was produced.

3.1 Scope of the Deliverables

Each deliverable is scoped to its purpose. The application note documents the hardware, so it reaches across every STM32 family that carries the trace unit. The proof of concept only has to establish feasibility, so it is held to the smallest configuration that proves it. The debugger has to serve daily work, so it carries the conventional debug operations alongside trace. Table 3.1 states where each boundary falls.

Table 3.1: Scope of the three deliverables

Deliverable	Scope
Application note	Instruction trace infrastructure of every STM32 family that implements it
	Keil μ Vision with the ULINKpro probe
	IAR Embedded Workbench with the I-jet Trace probe
Proof of Concept	ETMv4 implementations
	On-chip trace sink
	Instruction trace
Debugger	STM32 MCUs
	Breakpoints and variable inspection
	Peripheral inspection, memory views and disassembly
	Instruction trace

3.2 Requirements Analysis

This section states the requirements the work must satisfy. It separates them into functional requirements, which state what the solution does, and non-functional requirements, which state the conditions it operates under, and it then traces each requirement to the deliverable that carries it.

3.2.1 Functional Requirements

The functional requirements follow one trace from the hardware to the source line, and Table 3.2 keeps that order. FR1 to FR7 are the trace pipeline and hold the contribution of the project. FR8 to FR13 are the conventional debug operations, present so that trace joins a working debugger. FR7 keeps a gap in the capture visible in the record, which the trace data model and the frontend must both honor later.

Table 3.2: Functional requirements

ID	Requirement
FR1	Configure the trace unit and the on-chip trace sink of the target through the debug connection.
FR2	Start and stop trace capture in step with the run state of the program, without a separate command from the developer.
FR3	Extract the captured trace over the same debug connection that carries every other debug operation.
FR4	Decode the extracted byte stream into trace elements.
FR5	Reconstruct the sequence of executed instructions from those elements together with the program image.
FR6	Attribute each reconstructed instruction to a function, a source file and a line.
FR7	Represent a discontinuity in the capture explicitly, keeping the two sides of it distinct.
FR8	Present the reconstruction inside a development tool during a live debug session.
FR9	Provide the run control primitives of Section 2.2, namely halting, resuming, stepping, breakpoints and watchpoints.
FR10	Read and write memory and processor registers.
FR11	Present the registers of a peripheral by name and by field, using a description of the device.
FR12	Provide disassembly and source-level inspection of variables.
FR13	Reset the target and restore the debug session to a state consistent with the reset device.

3.2.2 Non-Functional Requirements

The non-functional requirements are the constraints of Chapter 1 stated as obligations on the solution. NFR1 has the largest effect on the design, since holding the hardware to the board's own probe leaves the on-chip buffer as the only trace route. NFR2 and NFR3 bind the result to both host platforms and to an existing development tool. Table 3.3 states them.

Table 3.3: Non-functional requirements

ID	Requirement
NFR1	Require no hardware beyond the debug probe already present on the development board, and no trace pins.
NFR2	Operate on Windows and on Linux.
NFR3	Integrate with existing development tools.

3.2.3 Traceability to Deliverables

Traceability confirms that every requirement has a deliverable, and that the deliverables build on one another. The application note and the proof of concept rest on the same pipeline requirements, so the proof of concept demonstrates what the note describes. The debugger owns the operations that make the reconstruction usable, together with the two portability requirements. Table 3.4 gives the assignment.

Table 3.4: Traceability of requirements to deliverables

Deliverable	Functional	Non-functional
Application note	FR1 to FR5	NFR1
Proof of Concept	FR1 to FR8	NFR1
Debugger	FR8 to FR13	NFR2, NFR3

3.3 Hardware Environment

This section describes the hardware on which the work was developed and tested. It presents the target microcontroller, the development board that carries it, the debug probe that connects the board to the host, and the trace components the target makes available.

3.3.1 Microcontroller: STM32H7S3L8

The main device used during the development period of the project is an STM32H7S3L8, a member of the STM32H7RS family. It carries an ARM Cortex-M7 processor [29] running at up to 600 MHz with a memory protection unit, a double-precision floating-point unit and separate instruction and data caches of 32 kilobytes

each. The device provides 64 kilobytes of internal flash memory and 620 kilobytes of internal SRAM, and is supplied in a TFBGA225 package [30].

Figure 3.1 shows the debug and trace components of the device and the buses that connect them.

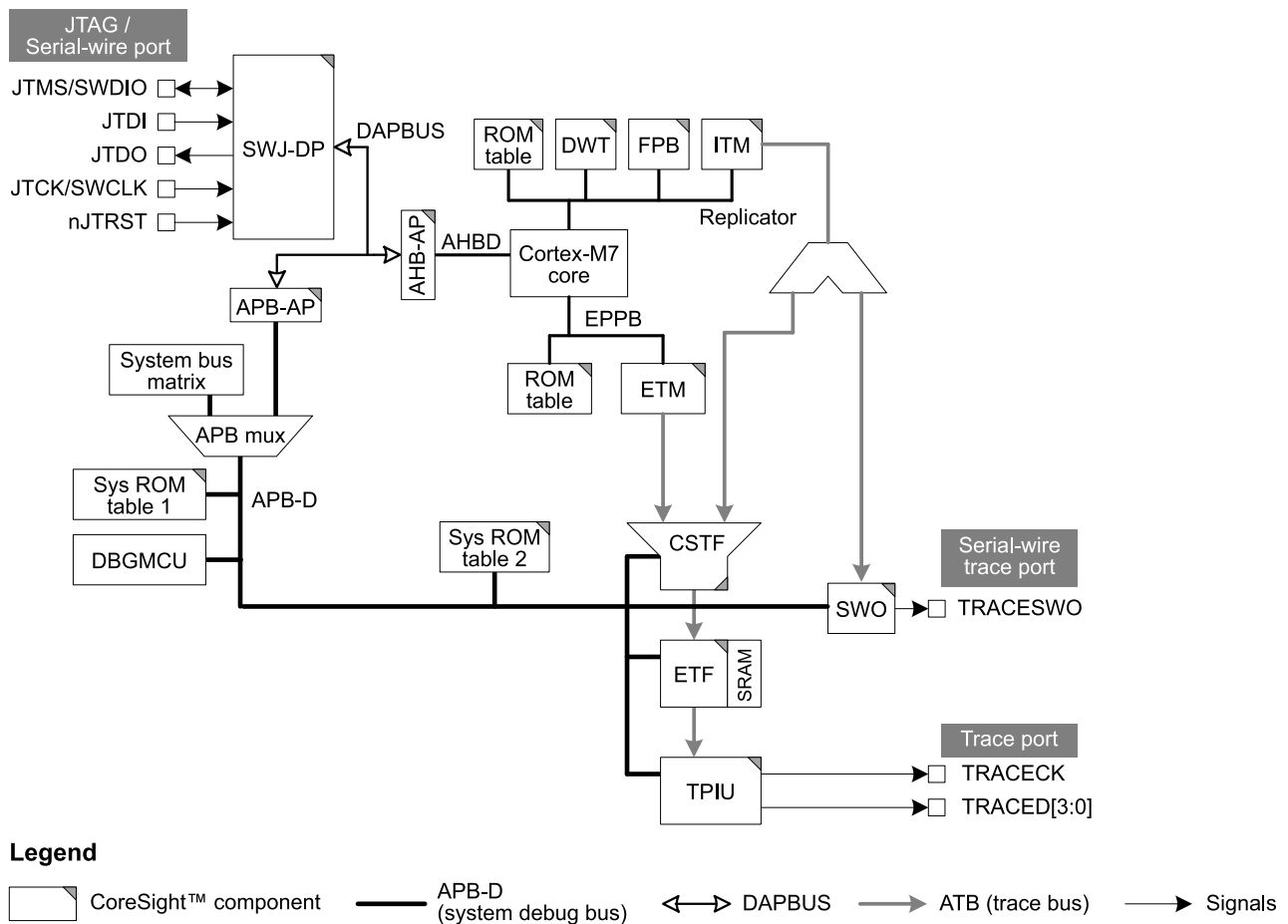


Figure 3.1: Debug and trace infrastructure of the target, adapted from [5]

3.3.2 Development Board: NUCLEO-H7S3L8

The microcontroller is used on a NUCLEO-H7S3L8. It exposes a MIPI20 connector for external debug tools, which carries the serial wire and JTAG signals together with a four-bit trace port [31].

3.3.3 Debug Probe: STLINK-V3EC

The probe is the STLINK-V3EC embedded on the board, which supports both SWD and JTAG [31]. It connects to the host through a USB Type-C connector and additionally provides a virtual COM port (VCP) and mass storage programming.

3.3.4 Trace Resources Available on the Target

These four components form the on-chip trace path of the target. The ETM emits the instruction trace, the funnel carries it toward the sink, and the TMC holds it in two kilobytes of on-chip memory. That capacity is

the limit against which every capture in this work is measured. The TPIU drives the external trace port, which this work leaves unused. Table 3.5 lists them with their base addresses.

Table 3.5: Trace components of the target device

Name	Base address	Note
ETM	0xE0041000	Implementation of the Cortex-M7 ETM, configured for instruction trace only [32].
TMC	0xE00F4000	ETF configuration, providing two kilobytes of memory.
TPIU	0xE00F5000	-
Funnel	0xE00F3000	Funnels the ETM trace data into the TMC. It also carries the stream of the instrumentation macrocell [5], which this work does not use.

3.4 Software Environment

This section describes the software used to build the work. It presents the development environment, meaning the languages, the compiler and the build tools, and then the external components the work depends on.

3.4.1 Development Environment

The engine is written in C++23 and the frontend in TypeScript, one language for each of the two programs. The same build runs on Windows and on Linux, so neither platform carries a separate procedure. Table 3.6 lists the tools.

Table 3.6: Development environment

Element	Choice
Host platforms	Linux and Windows
Backend language	C++23
Frontend language	TypeScript
Compiler	Clang
Build system	CMake, Ninja generator
Frontend bundler	esbuild
Editor	Visual Studio Code

3.4.2 External Software Components

The project adopts existing software for the parts that are not specific to it and builds only the parts that are. The conventional debugger, the on-chip access, the trace decoding, and the XML parsing come from the components

in Table 3.7. What sits between them, the trace drivers and the reconstruction pipeline, is the work of the project. Each entry carries the version used and the reason it was chosen.

Table 3.7: External software components

Component	Version	Description	Justification
OpenOCD	Fork of 0.12.0	Open-source on-chip debugger [7].	Supplies the groundwork the trace drivers extend.
LLVM	22	Compiler infrastructure and debugger, of which the machine code layer [33] and the debugger [34] are used.	Supplies the core debug functionality and the analysis of the program image.
OpenCSD	1.8.3	Library for decoding CoreSight trace [1].	Simplifies the trace decoding process.
Xerces-C	3.2.4	XML parser.	Reads the CMSIS-SVD device descriptions.
CodeSynthesis XSD	4.2.0	Generator of C++ bindings from an XML schema.	Avoids writing the CMSIS-SVD bindings by hand.
Visual Studio Code	1.125	Editor hosting the frontend extension.	Already implements the client side of the Debug Adapter Protocol.

3.5 Conclusion

This chapter has stated the scope of the work, the requirements it must satisfy, and the environment in which it was produced. The hardware and the requirement set both bear on what follows.

The hardware constrains which trace route is usable. The target implements both routes by which trace leaves a processor, and the bench can use only one of them, since the probe available cannot capture a trace port. The on-chip sink is therefore the route the work takes, and its capacity of two kilobytes is the constraint against which the results are read.

The requirement set fixes the shape of the work. The functional requirements describe a pipeline running from the configuration of a trace unit to the display of an executed source line, and the non-functional requirements constrain it to hardware already present and to an existing development tool. Chapter 4 describes the structure built to satisfy them.

4 Architecture

Introduction

This chapter states the structure of the solution. It names the components, states what each is responsible for, and states why the responsibilities were divided as they were. Implementation is the subject of Chapter 5.

4.1 System Overview

This section states the solution as a whole before the later sections detail its parts. It presents the global architecture, the capabilities the engine provides, and the design rules that every component observes.

4.1.1 Global Architecture

The solution is an embedded debugging engine paired with a frontend. The engine performs every debug operation. The frontend presents the results to the developer. The two are separate programs joined by the Debug Adapter Protocol.

The alternative was a single program with the interface built in. Two programs were chosen because the result must plug into a development tool the developer already uses, and a built-in interface would bind the engine to one tool. Kept separate, the engine is reached by any frontend that speaks the Debug Adapter Protocol, and it runs without a frontend for offline work.

The engine connects to the target device through a debug probe. The probe is reached from the host over USB and connects to the device over SWD. Instruction trace is captured in an on-chip buffer and read back over the same debug connection. No trace port and no trace-capable probe are required.

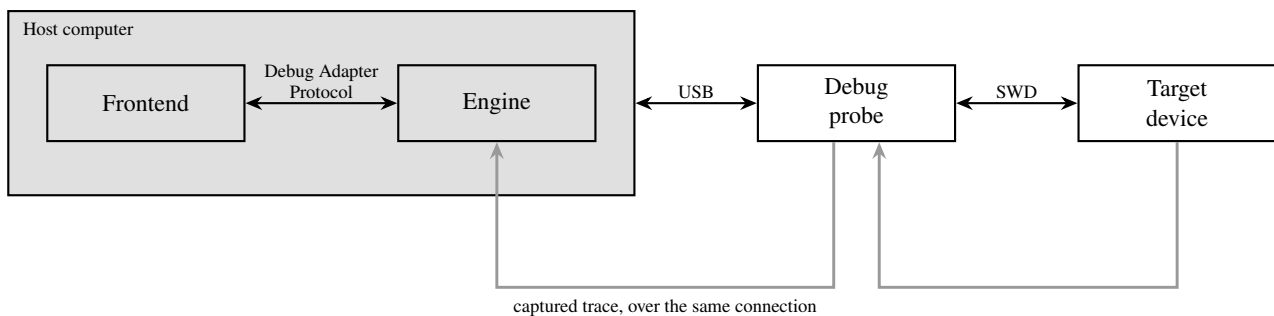


Figure 4.1: Global architecture of the solution

4.1.2 Debugger Capabilities

The engine provides the following capabilities, grouped by area.

- **Run control** — breakpoints, stepping, pause, reset
- **Inspection** — variables, expressions, stack frames, disassembly
- **Memory** — read, write, live observation
- **Peripherals** — device description, register access, live observation
- **Trace** — instruction trace capture, decoding, reconstruction, source correlation
- **Symbols** — modules, source resolution

Capabilities the standard protocol does not cover are exposed through its extension mechanism.

4.1.3 Design Rules

- No component of the engine names an interface of a particular development tool. A second frontend can be written against the same engine without change.
- A failed operation produces an error response and the process continues. A device may be absent, in reset, or unresponsive.
- Components that produce output publish typed domain events. A single subscriber converts each event into a protocol message. No component constructs protocol messages directly.

4.2 Debugger Architecture

The engine is divided into six layers. Each depends only on the layers below it. The protocol types are a shared dependency used by the handlers and the context.

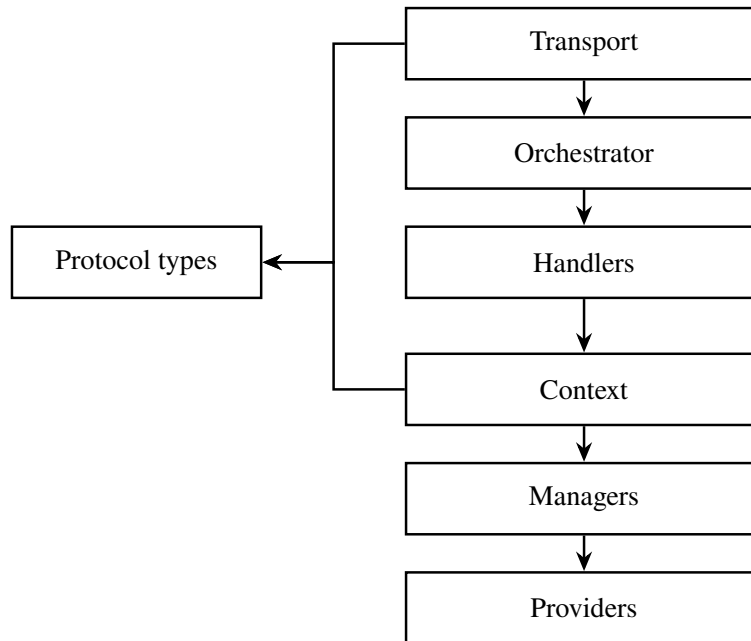


Figure 4.2: Module dependency graph

4.2.1 Transport Layer

The transport is the outer boundary of the engine. It reads and writes framed messages on a pair of channels. It performs no interpretation of message content. It is not bound to a particular protocol or a particular channel type.

4.2.2 The Event Bus

The event bus is a typed publish-subscribe mechanism. A producer publishes a domain event describing what happened. A single subscriber converts each domain event into a protocol message and sends it through the transport. Every producer stays free of knowledge of the wire format.

4.2.3 Orchestration Layer

The orchestrator receives messages from the transport and routes each to the handler responsible for it. It consumes events from the event bus and sends them outward through the transport. It tracks operations in progress and supports their cancellation.

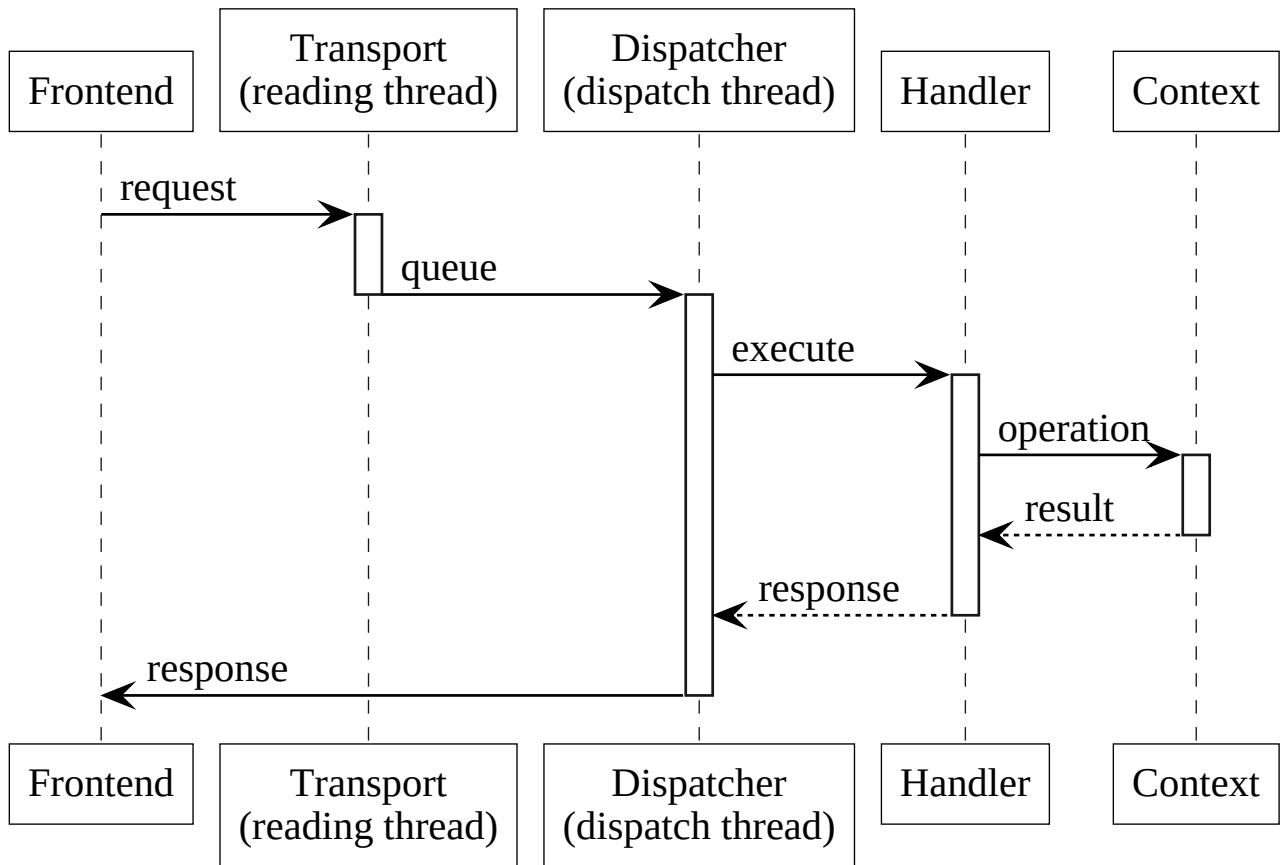


Figure 4.3: Request flow from transport to response

4.2.4 Request Handlers

The handler interface defines the contract for fulfilling a single request. A handler receives a parsed request and produces a response or an error. Handlers may respond synchronously, completing before the next request is dispatched, or asynchronously, deferring their response while the engine continues to accept other messages.

4.2.5 The Debug Context

The debug context is a facade. It composes the event bus, the shared providers and the managers, and every handler holds a reference to it. Managers reach the shared providers and one another through it.

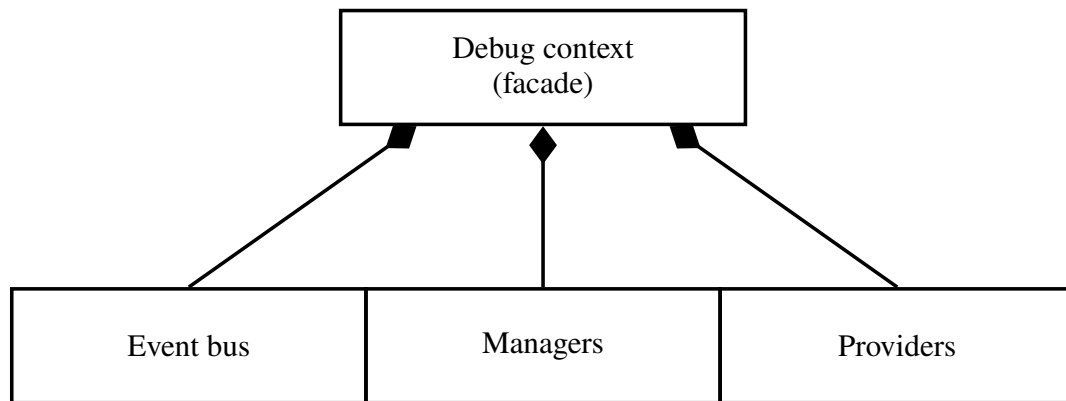


Figure 4.4: The debug context

4.2.6 State Managers

Each manager owns one area of debug state: breakpoints, memory, disassembly, modules, variables and expressions, execution control, session lifecycle, live observation, device and peripherals, and instruction trace. A manager that needs a dedicated external resource owns the provider for it. All managers use the shared providers through the debug context.

4.2.7 External-System Providers

A provider adapts an external system to the needs of the engine, and no type belonging to an external system appears in any provider interface. Without this rule the types of OpenOCD, LLDB and OpenCSD would spread through the engine, and every component that touched them would depend on the library that defined them. The alternative was to let each component call those libraries directly. The abstraction was kept for three reasons. A component is tested against a substitute provider with no device and no library present. A library is replaced behind its provider without reaching the components above it. The core of the engine states its needs in its own terms and stays independent of any external interface. Five providers exist.

Table 4.1: Provider categories

Provider	Responsibility
Debug primitives	Conventional debugging: symbol resolution, expression evaluation, stepping, stack unwinding, breakpoint management
Extended capabilities	Direct device access: memory read and write, command execution, trace component management, target state observation
Device	Device description: memory map and peripheral model
Disassembler	Static program analysis: load a program image, produce the precomputed instruction table
Trace	Trace decoding and reconstruction: accept a captured block, return the increment

4.2.8 Concurrency Model

The engine is multi-threaded. Three requirements drive the model.

- A cancellation must be received while a request is being handled.
- Observation of memory and peripherals must continue while the program runs.
- The debug primitives provider does not support concurrent use.

A reader thread receives messages from the transport. A dispatch thread executes them one at a time. Separating the two allows a cancellation to be recorded as soon as it arrives.

The extended capabilities provider confines its operations to a dedicated thread. Observation workers access it directly, which is what allows observation to proceed independently of request handling.

Access to the debug primitives provider is serialized. A single operation accepts a callable and runs it under a lock.

Every background thread that produces output publishes a domain event. The single subscriber converts each event into a protocol message.

4.3 Instruction Trace Architecture

This section states the pipeline that turns the execution of a program into a record expressed in source terms. It is the primary contribution of the project.

4.3.1 Pipeline Stages

Trace leaves the processor as a compressed packet stream. The pipeline has four stages.

1. **Generation and capture.** The trace source is configured to emit packets and the trace sink is configured to store them. Both are coupled to the run state of the processor.
2. **Extraction.** The captured data is transferred from the on-chip buffer to the host when the processor halts.
3. **Decoding.** The compressed byte stream is converted into trace elements.
4. **Analysis.** Trace elements are resolved against the program image and its debug information to produce executed instructions, each attributed to a source location.

The stages are separated because each depends on a different thing. Generation and capture depend on the device, extraction on the debug connection, decoding on the trace protocol, and analysis on the program image. A stage that combined two of these could run only when both were present.

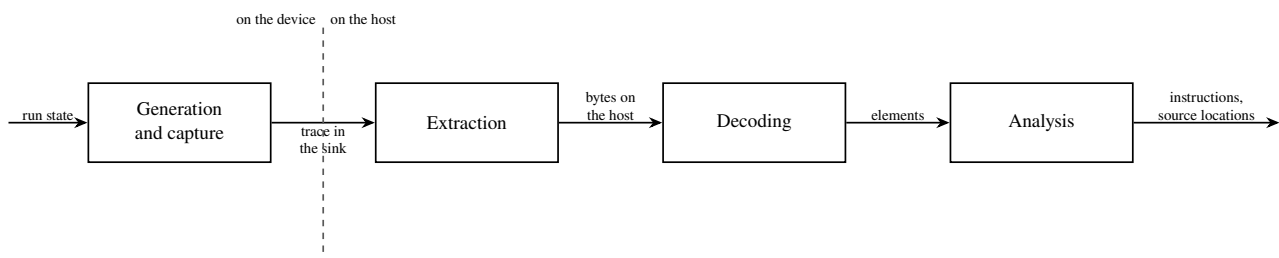


Figure 4.5: The four-stage instruction trace pipeline

The separation has four consequences. The work that needs the device is held apart from the work that does not, so the two do not interleave. The last two stages run on a stored capture, so a trace is analyzed offline with no device present. Each stage is tested on its own against fixed data. A stage is reused where its inputs allow, since decoding and analysis depend on the capture and the image alone.

4.3.2 Trace Data Model

The stages exchange value types. A reconstructed instruction records one executed instruction. A function block groups consecutive line blocks belonging to one function, and a line block groups consecutive instructions sharing one source location. A gap records a discontinuity and carries the reason for it.

4.3.3 Trace Lifecycle

Trace progresses through four states: unavailable, idle, armed, and capturing. The diagram below shows the transitions.

The distinction between armed and capturing exists because configuration must complete before execution begins. The pipeline is prepared while the processor is halted, so that no trace is lost at the start of a run.

Each halt produces one capture. A session that resumes and halts repeatedly accumulates a sequence of captures, each appended to the record. A reset discards the entire record, because it describes a program run that no longer exists.

Two kinds of discontinuity appear in the record. A buffer overflow during a run causes the oldest data to be lost. A halt between two captures is itself a gap, since nothing is recorded while the processor is stopped.

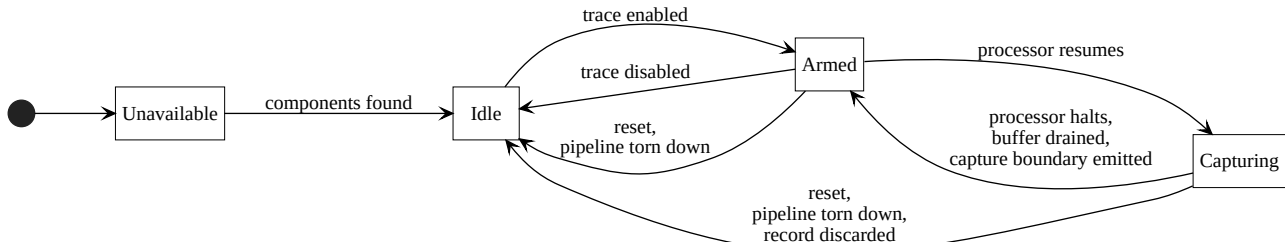


Figure 4.6: The trace lifecycle

4.4 Frontend Architecture

The frontend is an extension for a development tool. A session object tracks which capabilities the engine advertised and enables the matching views. Each view holds a model that accumulates what the engine sends and a presentation that renders it. The engine sends increments, so the cost of an update follows the size of the change.

4.5 Conclusion

The instruction trace pipeline is the contribution of the project. Its stages are separated so that each depends on one thing, which is the device, the extended capabilities provider, the trace protocol, or the program image.

The engine that carries it is structured around provider abstraction, event-driven communication, and a transport and orchestrator pair that decouples it from any specific protocol or frontend.

Consequences of that arrangement recur in the next chapter. Observation continues while the program runs, which makes live memory and peripheral views possible. Reset is performed outside the conventional path, which makes resynchronization necessary. Chapter 5 describes how each was realized.

5 Implementation

Introduction

This chapter states how the architecture of Chapter 4 was realized. It begins with the methodology by which the work was carried out, then treats the external systems the engine embeds, the trace pipeline, the debugger and the frontend.

The engine builds on existing open-source work, and the boundary between what was adopted and what was written is stated here at the outset. OpenOCD, LLDB and OpenCSD are adopted. Written for this project are the two CoreSight drivers, the trace pipeline from extraction through reconstruction, the domain components at the core of the engine, the device description provider, and the frontend extension.

5.1 Development Methodology

The work was built in stages, each chosen so that a part could be validated before the next depended on it. Several architecture decisions of Chapter 4 were reached this way, by finding that an earlier arrangement did not hold. The order followed the trace itself, from the parts that need no device to the parts only hardware can exercise.

The first stage explored the trace infrastructure and produced the application note of Section 5.2. It also produced an offline tool that decoded a stored capture against a program image, which established the decoding and reconstruction stages with no device present. Beginning offline let these stages be validated against fixed data before any acquisition existed.

The second stage addressed acquisition. Obtaining a capture without manual intervention required the trace components on the device to be configured and drained, which OpenOCD did not support, so the drivers of Section 5.3.2 were written.

The third stage integrated the pieces. The offline tool was absorbed into a debug engine and OpenOCD was linked into it, which turned a mechanism that printed a reconstructed trace into one usable during a debug session.

The fourth stage added the remaining debugger capabilities and the frontend, and it hardened the result against a real device, where the defects of Section 6.7 appeared. The fifth stage addressed portability and

packaging.

Figure 5.1 shows the stages and their dependencies.

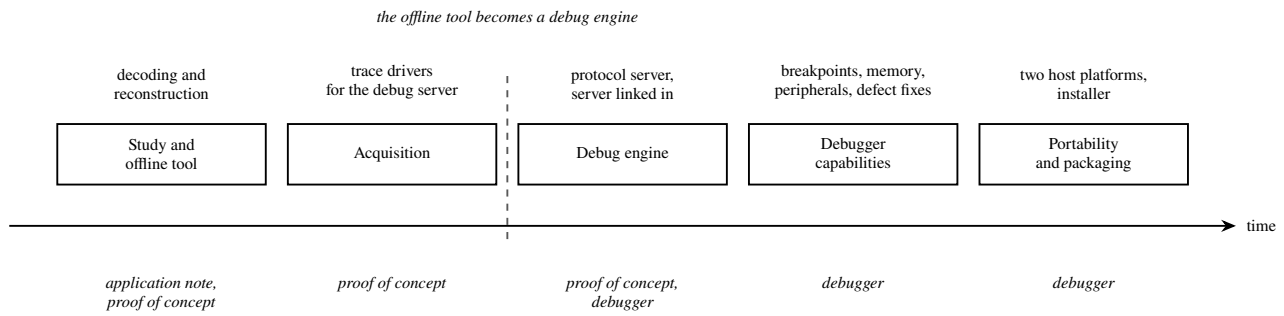


Figure 5.1: The development methodology

5.2 The Application Note

The information required to use instruction trace on these devices is correct, complete and distributed across documents written for different readers. A developer must assemble a procedure from several documents, and the assembly is the difficult part. The audience is an embedded developer who has a device, a probe and a debug session, and who wants execution history.

The first part surveys the trace infrastructure of the devices and restates in one place what the specifications and the reference manuals state separately. A reader can determine whether the trace capability is reachable with the hardware already available.

The second part walks through configuring the trace source and the trace sink, arming them, obtaining a capture, and decoding it against the program image. The difficulty of the task lies in the ordering and in the preconditions, and the walkthrough states each condition at the point where it applies.

5.3 On-Chip Debugger: OpenOCD

OpenOCD is a standalone program. Making it callable from the engine required converting it into a library and adding drivers for the CoreSight components the project requires. The conversion and the two drivers are the work of this project, and OpenOCD's own debug functionality is used unchanged. This section covers them.

5.3.1 Embedding as a Library

OpenOCD is designed to run as a self-contained process. Embedding it inside the engine required four changes.

5.3.1.1 Build System Migration

The original build system was replaced so that OpenOCD could be compiled as a static library consumed by the engine. Each group of source files became a CMake object library, and those groups were aggregated into a

single library target with no entry point. The source file containing `main()` remains in the tree and is excluded from the build.

5.3.1.2 Containing Process Termination

OpenOCD calls `exit()` in many error paths. A library must not terminate the host process, since a failure affecting one device must not end the debug session.

Every `exit()` call was redirected to a replacement function, imposed on each translation unit by the build. The replacement checks a per-thread guard. When a guarded call is in progress, the replacement performs a non-local jump (`setjmp/longjmp`) back to the guard site, which returns the termination code to the caller as a failure. When no guard is active, the replacement terminates as before.

Every entry into OpenOCD from the engine is wrapped in such a guard. Destructors on the abandoned call chain are not run, and allocations are leaked. The mechanism is a containment of last resort.

5.3.1.3 Embedding the Startup Scripts

OpenOCD reads Tcl scripts from disk at startup. These scripts define much of its command surface. They are converted into byte arrays at build time and compiled into the library, removing the file-system dependency.

5.3.1.4 Runtime Initialization

The initialization that OpenOCD's `main()` would have performed is reimplemented by the engine. A command context is created, the logging subsystem is started, and each subsystem's command set is registered in the order OpenOCD requires, since later registrations depend on earlier ones.

The startup sequence then follows the order OpenOCD requires: initializing targets, the adapter and the transport, initializing the debug access port, examining the targets, initializing the flash and trace subsystems, and starting the GDB RSP server.

5.3.1.5 Thread Affinity and Command Serialization

Every OpenOCD operation executes on the thread running its server loop. That loop blocks in `select()` over sockets it owns. Work submitted from another thread must reach that thread and must wake the loop.

A service is registered with OpenOCD, bound to a port on the loopback interface, and a client socket is connected to it. Sending one byte on that connection wakes the loop. The service handler drains a queue of pending work and executes each entry. Every public method of the OpenOCD provider submits work to that queue and waits for the result, with a bounded timeout so that no caller blocks indefinitely.

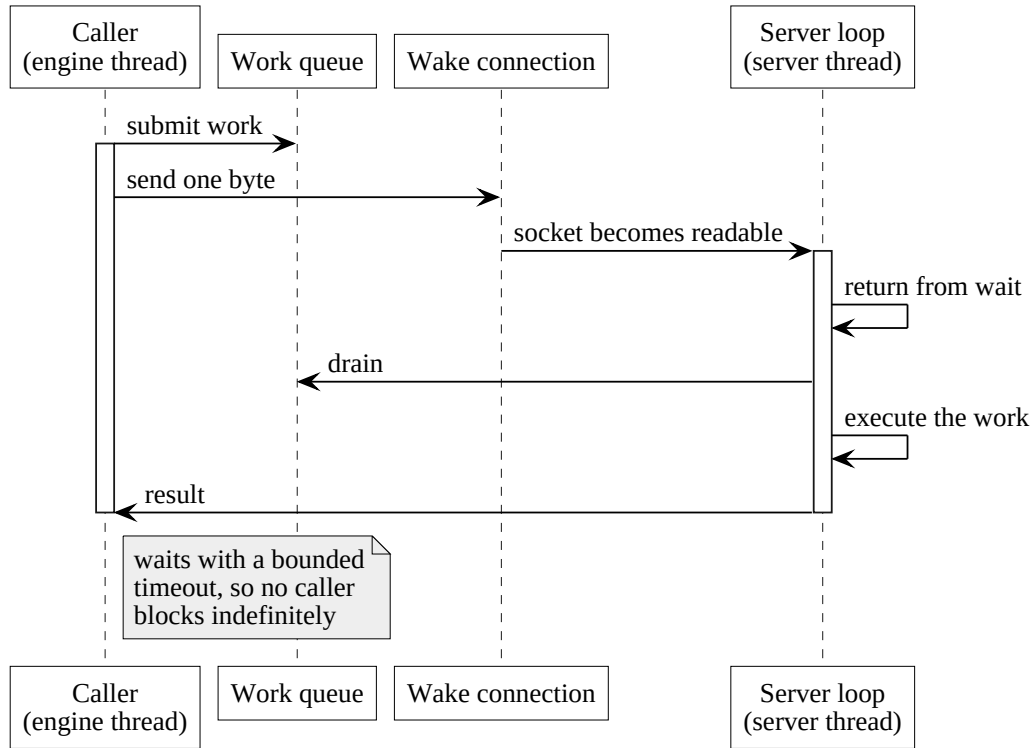


Figure 5.2: Command queue and wake mechanism

5.3.2 CoreSight Drivers

OpenOCD carries drivers for earlier CoreSight trace components but does not support the TMC or ETMv4. The two drivers were implemented as part of the project, following the patterns established by the existing CoreSight infrastructure.

Each driver is created as a named Tcl object within the OpenOCD configuration. The object exposes a `configure` command for staging options, an `init` command for hardware validation and initial setup, and `enable` and `disable` commands for requesting that trace follow the run state of the target.

5.3.2.1 TMC Driver

The driver handles all three TMC variants. The variant is determined at initialization from the `DEVID` and `DEVTYPE` registers.

The driver maintains an explicit state machine to track what the hardware is doing. Figure 5.3 shows the transitions. The state governs what operations are valid at any given moment and prevents redundant hardware access.

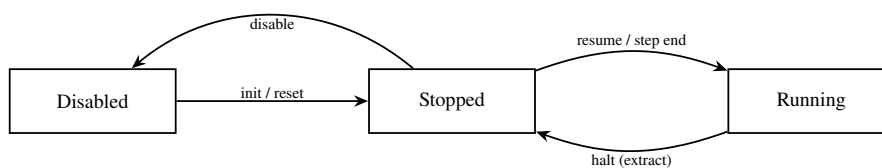


Figure 5.3: TMC state transitions

5.3.2.2 ETMv4 Driver

The ETMv4 driver adapts to the particular implementation present on the device. At initialization it reads all TRCIDR registers and decodes them into a capability structure.

First, each configuration request is validated against the discovered capabilities. Second, the driver applies capability-dependent defaults: features that must be enabled when supported are turned on, and features that may degrade gracefully are adjusted. Third, the raw identification register values are preserved as a snapshot for the trace decoder.

5.3.2.3 Configuration Model

Both drivers follow a stage-validate-commit model. Options are staged by the `configure` command, which sets dirty flags without touching hardware. When the commit path runs, it validates the staged options against the discovered identity and capabilities, then writes only the dirty registers as a batch of fixed-order, queued DAP writes. The fixed order is necessary because certain registers become meaningful only after others have been written.

5.3.2.4 Run-State Coupling

Both drivers register callbacks on target events and act on processor state transitions.

On resume, an already enabled driver validates, commits and starts the component. On halt, the ETMv4 driver stops the trace unit and the TMC driver stops capture and drains the buffer. When stepping, the ETMv4 driver starts before the step and the TMC driver starts after it, keeping the sink disarmed during single-step execution. On reset, both drivers recommit their full configuration, since the reset invalidates all register values.

Trace therefore follows the program with no host action during a run.

5.3.3 Trace Extraction

Extraction runs when the processor halts.

Capture is stopped before it is read. The driver requests that the component stop on flush, issues a manual flush, then waits for the `TMCReady` status bit. A component reporting an empty buffer is handled separately.

The size of the buffer to read is obtained from the `CBUFLEVEL` register if the component doesn't report an overflow of the buffer, otherwise, the entirety of the on-chip buffer is read. The fill level is meaningful only while capture is enabled, so capture is disabled only after the drain has completed.

The buffer is drained through `TMC_RRD`, a single register that returns successive words. The read is issued as a repeated access to the register address with the automatic address increment of the access port suppressed.

The extracted block is delivered to a callback registered by the engine and, where configured, to a file or a TCP connection. A full synchronization frame is emitted ahead of every payload.

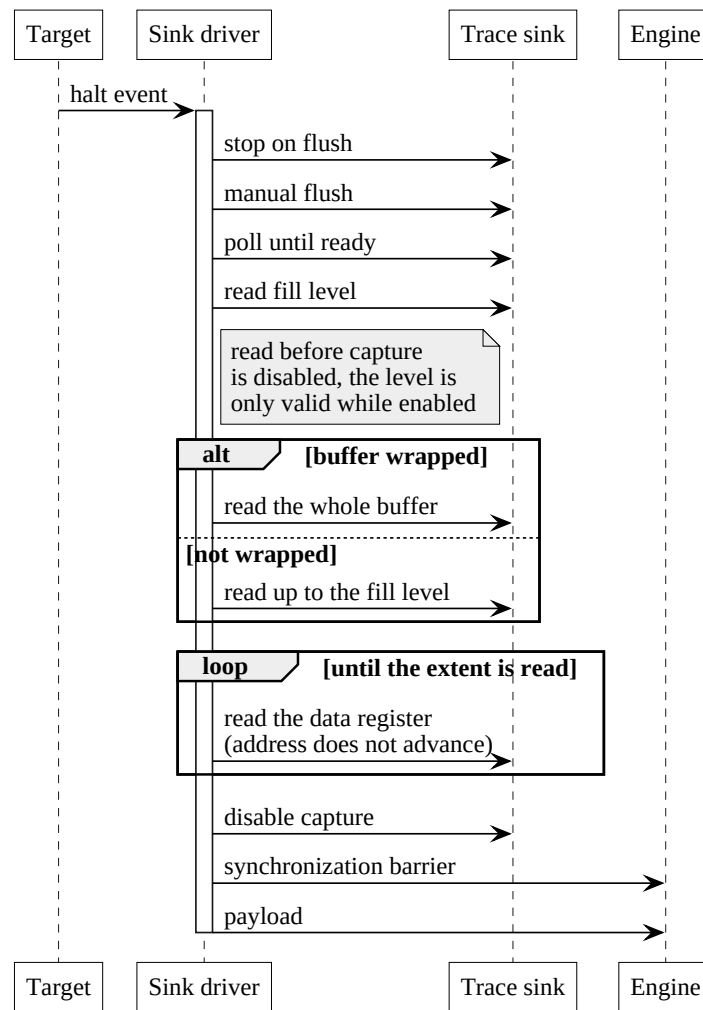


Figure 5.4: Extraction sequence

5.4 Source-Level Debugging through LLDB

The engine drives LLDB through the defined LLDB SB types.

LLDB connects to the target over GDB RSP, served by OpenOCD inside the same process. The connection is made over the loopback interface.

Serialization of target access acquires the recursive API mutex of the target and invokes the supplied callable.

At startup, the following configuration elements are applied to the LLDB instance:

- External symbol lookup is disabled, since LLDB otherwise attempts to fetch symbols from network services.
- Parallel module loading is disabled, since the background workers it creates acquire locks in an order opposite to the serialization above.

- The strategy for resolving breakpoints at inlined call sites is fixed, preventing a developer's personal configuration from changing where breakpoints are placed.

OpenOCD routed every GDB RSP packet beginning with a particular letter to a handler for an obsolete feature, while LLDB sends several unrelated packets beginning with that letter. OpenOCD was modified to match the complete packet name and replies to the others with the empty response that indicates an unsupported packet.

5.5 Division of Work

The engine drives two external systems that control the target device, and this section describes the responsibilities of each system.

Neither system covers the work on its own. LLDB provides source-level debugging, the DWARF and expression machinery, and stack unwinding, but it reaches the device only through GDB RSP, which offers no path to the trace components and no access to memory while the processor runs. OpenOCD reaches the debug port, the access ports and the trace components directly, but it carries no model of the program and no source-level debugging. Combining the two gives the engine both, and the responsibilities below follow from that division.

Debug primitives mentioned in Section 2.2 are handled by LLDB.

The decoder requires typed register values read from hardware, and a capture arrives as a push at the moment of a halt. The engine therefore also calls OpenOCD directly through its provider.

Reading memory through LLDB requires a stopped program. A direct read through OpenOCD reaches the device while the processor runs allowing for live memory watches.

Memory access is selected when the session is established. A launched session installs the direct path through OpenOCD, and every read and write uses it. An attached session has no direct path, so trace, live memory observation and peripheral inspection are unavailable.

Reset is performed on the direct path. Section 5.7.2 describes why.

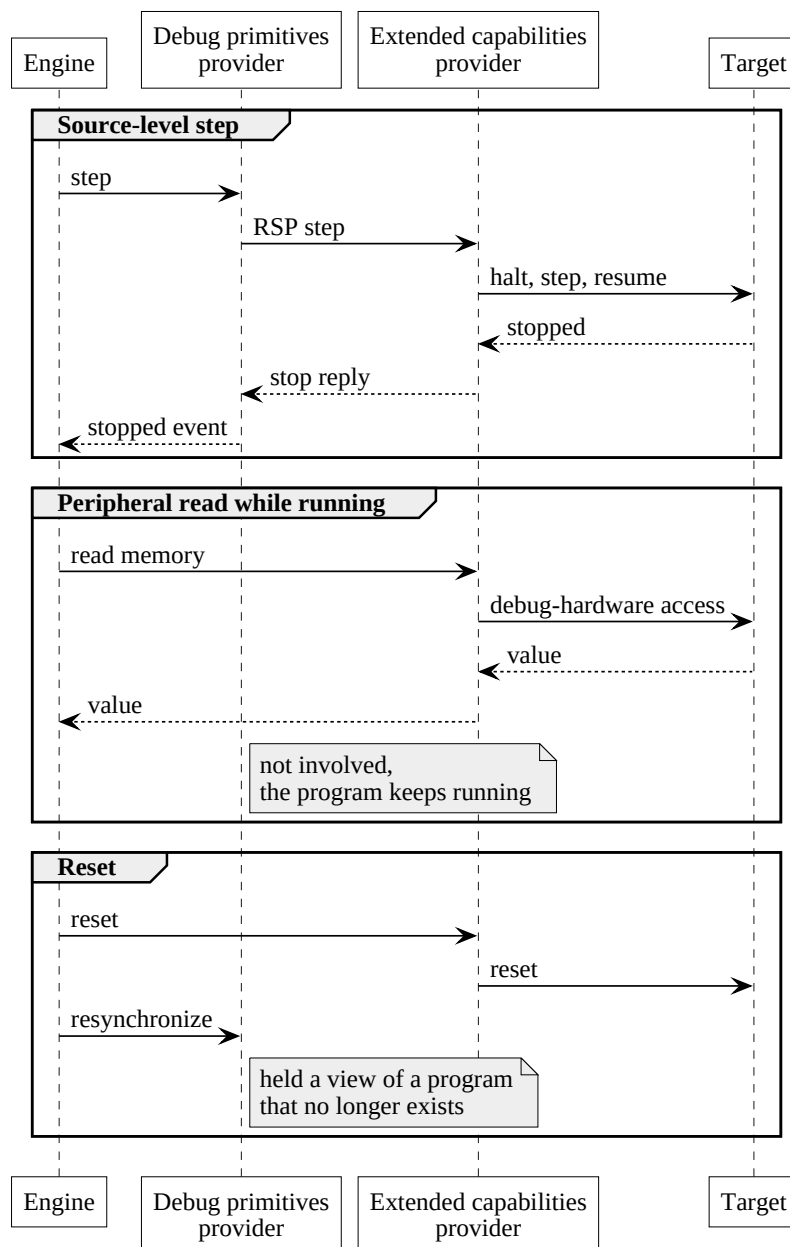


Figure 5.5: Division of work between LLDB and OpenOCD

5.6 The Trace Pipeline

The trace pipeline operates in four stages: decoding, static program analysis, reconstruction and delivery.

5.6.1 Decoding

Decoding is performed by OpenCSD. The implementation consists of configuring it correctly for each capture.

A decode tree is constructed for each block, setting up frame formatting, memory alignment, and resets on full sync frames.

The decoder must be configured exactly as the trace unit was. The identification registers are read from the unit itself, so the decoder is configured against the capabilities the hardware reports. The control register

and the trace identifier hold the values the driver committed.

The program image is registered with the decoder as the source of instruction bytes, described by its loadable segments.

The decoder reports elements to a sink record object. The sink retains instruction ranges, exception and exception return elements, trace-on elements and loss-of-synchronization elements, as well as the byte offset within the captured buffer, and the trace identifier of the stream.

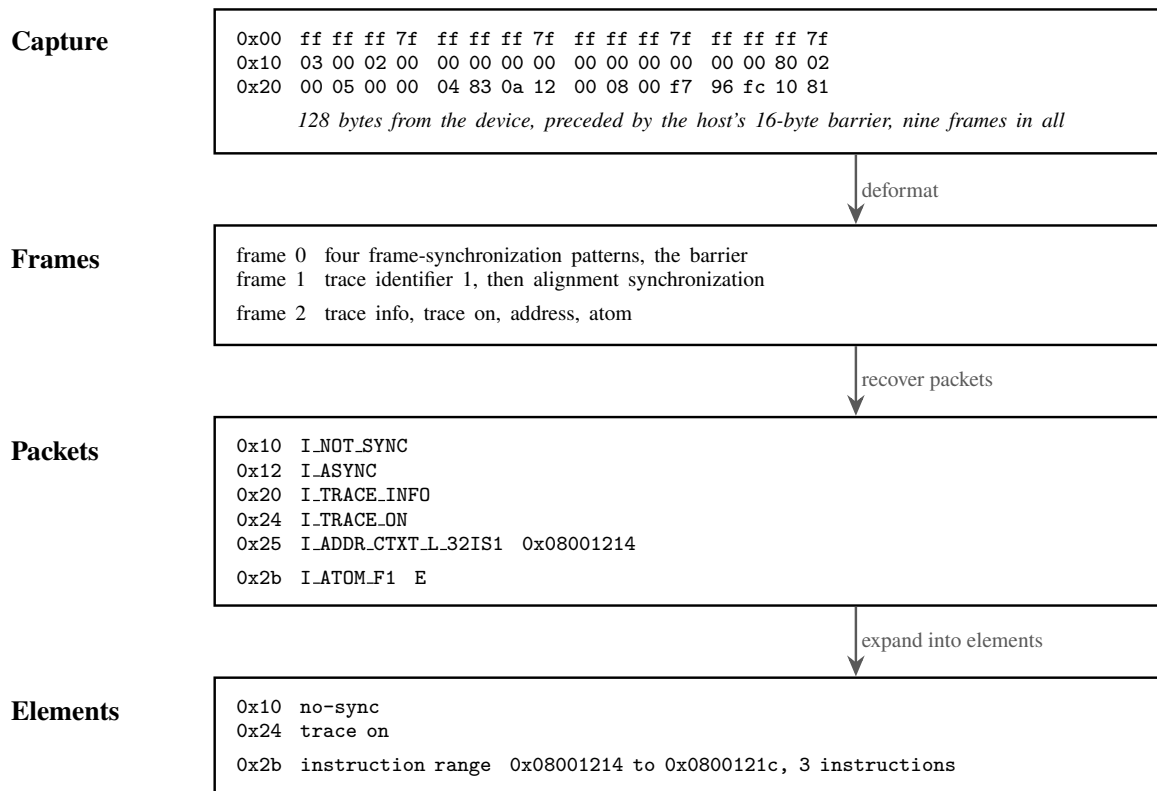


Figure 5.6: From bytes to elements, on the capture held as test data

5.6.2 Static Program Analysis

The program image is opened as an ELF object file via LLVM, and its loadable segments are extracted, and registered with the decoder.

The mapping symbols partition the image into marker spans, skipping data markers to prevent literal pools from being decoded as instructions. And treating sections with no mapping symbols as a single span of the default instruction set.

Each span is decoded instruction by instruction. For every instruction the analysis records its address, length, raw bytes, mnemonic, operands and a classification: branch, call, return, conditional or indirect. When a branch target is determinable from the instruction, it is recorded. LLVM's analysis facility does not recognize the instruction forms Thumb2 architecture emits for a return, so those forms are matched on the printed instruction.

The source location of every decoded instruction is resolved during the same pass to keep reconstruction

expensive, and it is done by requesting the whole chain of inlined frames.

5.6.3 Reconstruction

The decoded elements are converted into the model of Section 4.3 by transformations expressed over the same record sequence.

A range is expanded by walking from its start address to its end address, taking the length of each instruction from the precomputed table. When the decoder reports that the last instruction of a range was not executed, the transformation marks it and keeps it. The instruction is dropped one layer higher, when the increment is assembled, so the output of the transformation remains a complete account of what the decoder reported.

Consecutive executed instructions sharing an identical chain of source locations are merged into a line block, and consecutive line blocks belonging to the same outermost function are merged into a function block. Comparison uses the whole inlining chain to differentiate contiguous instructions from different call chains. Grouping is chronological and performs no deduplication.

Discontinuities are detected by maintaining a running count of executed instructions and emitting a gap wherever the element stream indicates one. The causes distinguished are buffer overflow, resumption of tracing after it had stopped, and loss of synchronization.

5.6.4 Delivery

The result is sent to the frontend as an increment. Each message states the index at which its contents begin, followed by the instructions, function blocks and gaps the capture produced.

5.7 The Debugger

This section describes how the debugger is implemented on the architecture of Chapter 4. It follows the session from its creation through the operations the developer drives, covering the session lifecycle, reset and resynchronization, breakpoints, memory and live observation, peripheral access, variables and expressions, and the extensions that carry the operations the standard protocol does not define.

5.7.1 Session Lifecycle

Launching creates the OpenOCD provider, installs the memory access strategy that uses it, creates the LLDB target from the program image, connects LLDB to OpenOCD's GDB RSP server, and starts the trace pipeline.

Attaching connects LLDB to a GDB RSP server the engine did not create. Every capability depending on the direct path is unavailable in this mode.

Disconnection proceeds in a fixed order. The program is detached or terminated, event handling is

stopped, the trace pipeline is shut down, the observation workers are stopped, and the OpenOCD provider is destroyed last, so that no thread outlives the resource it reads from.

5.7.2 Reset and Resynchronization

Resetting the device is the operation where LLDB and OpenOCD conflict.

The reset is performed through the OpenOCD provider, because the reset-and-halt behavior required is not reachable through GDB RSP. The processor therefore halts without LLDB being aware meaning that its cached view of threads, frames and variables is stale.

By default, LLDB is made aware of resets via a reply to a reset request it has made, and therefore a new mechanism needs to be introduced for asynchronous device resets.

A command was added to OpenOCD that arms a synchronization flag on the GDB RSP connection. When the flag is armed, the next resume or step request received on that connection is intercepted and answered with a report of the current state. The engine arms the flag after the reset and then issues a single-instruction step through LLDB. OpenOCD intercepts the step and answers allowing LLDB to update its view, and the session continues against the program that is actually present.

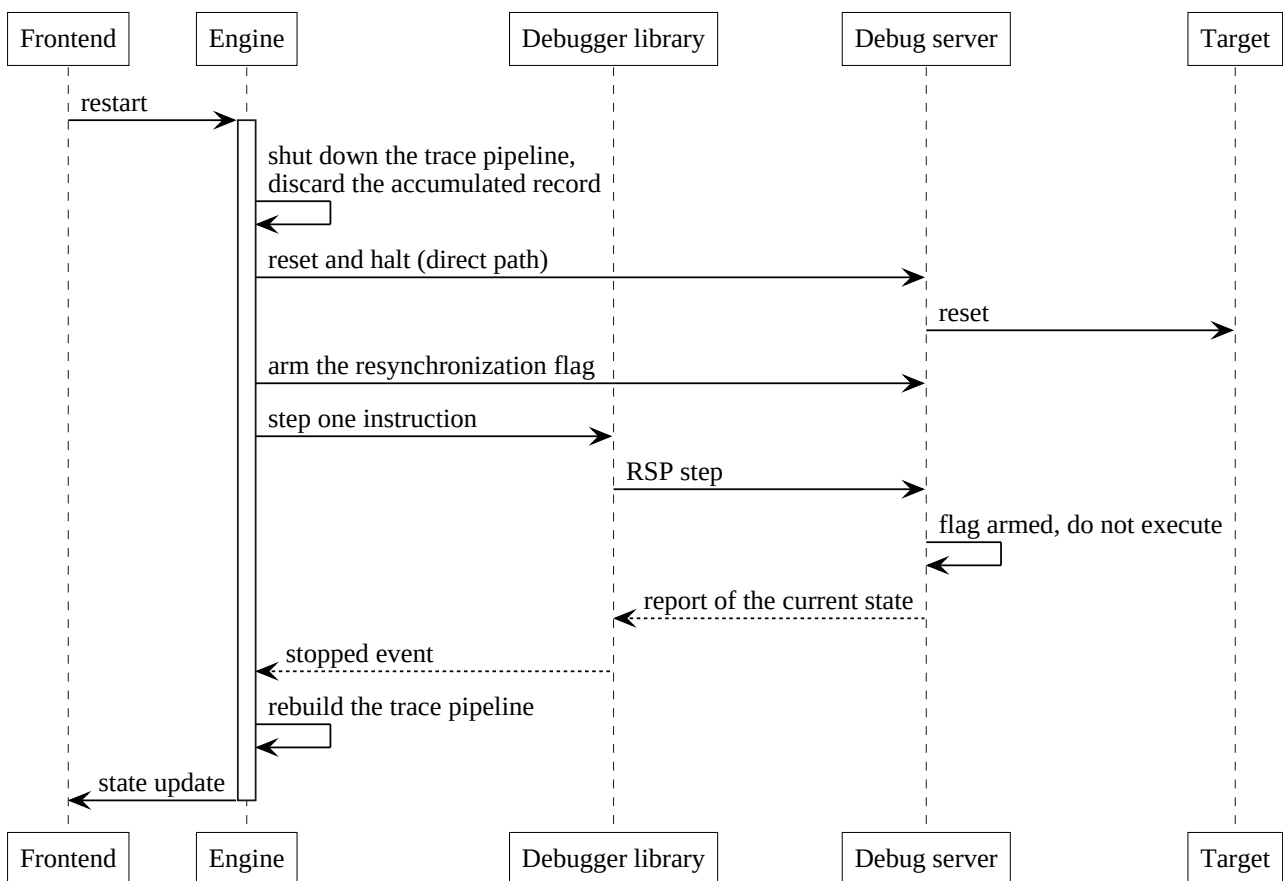


Figure 5.7: Reset and resynchronization

5.7.3 Breakpoints

The choice between a software and a hardware breakpoint follows from the memory region the address lies in. The engine makes that choice automatically.

For each breakpoint the engine consults the memory map provided by the device description. If the location lies in writable memory, a software breakpoint is used. Otherwise a hardware breakpoint is used. Where no memory map is available, a hardware breakpoint is used by default.

The developer may override the decision.

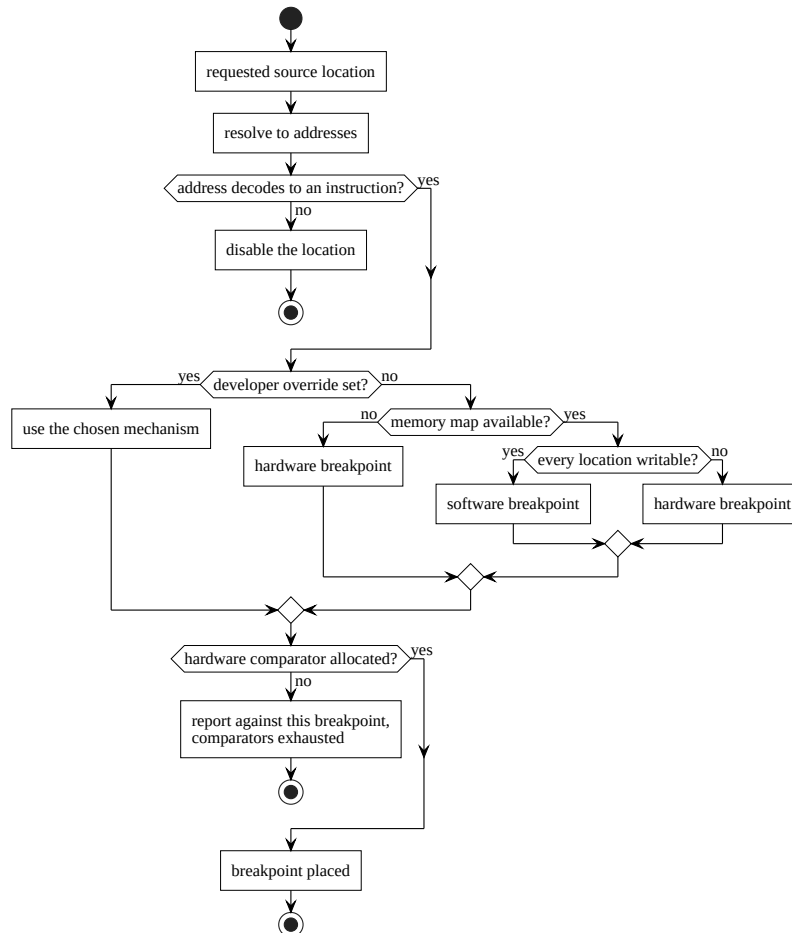


Figure 5.8: Breakpoint placement decision

5.7.4 Memory and Observation

Memory is read through one of two strategies, selected when the session is established. `ProcessMemoryStrategy` reads through LLDB and serves a conventional session. `OpenOcdMemoryStrategy` reads through the OpenOCD provider and is installed whenever the direct path is available.

The second strategy does not require the program to be stopped. Live observation is built on it. Each observation runs on a dedicated thread, reads the requested region at a configured interval, and publishes the result on the event bus. The wait between reads is interruptible so that stopping an observation takes effect

immediately.

Observations are not suspended while the processor is halted.

5.7.5 Peripherals

The device description is parsed from the vendor SVD file into a plain data model of peripherals, registers and fields.

A register whose reading changes the state of the peripheral is not read by default. Registers that are adjacent and safe to read are coalesced into a single access, reducing a peripheral view to a small number of reads.

A race between a write and a periodic read is resolved on the producing side. A counter is advanced whenever a write completes, and an observation whose read overlapped a completing write is discarded.

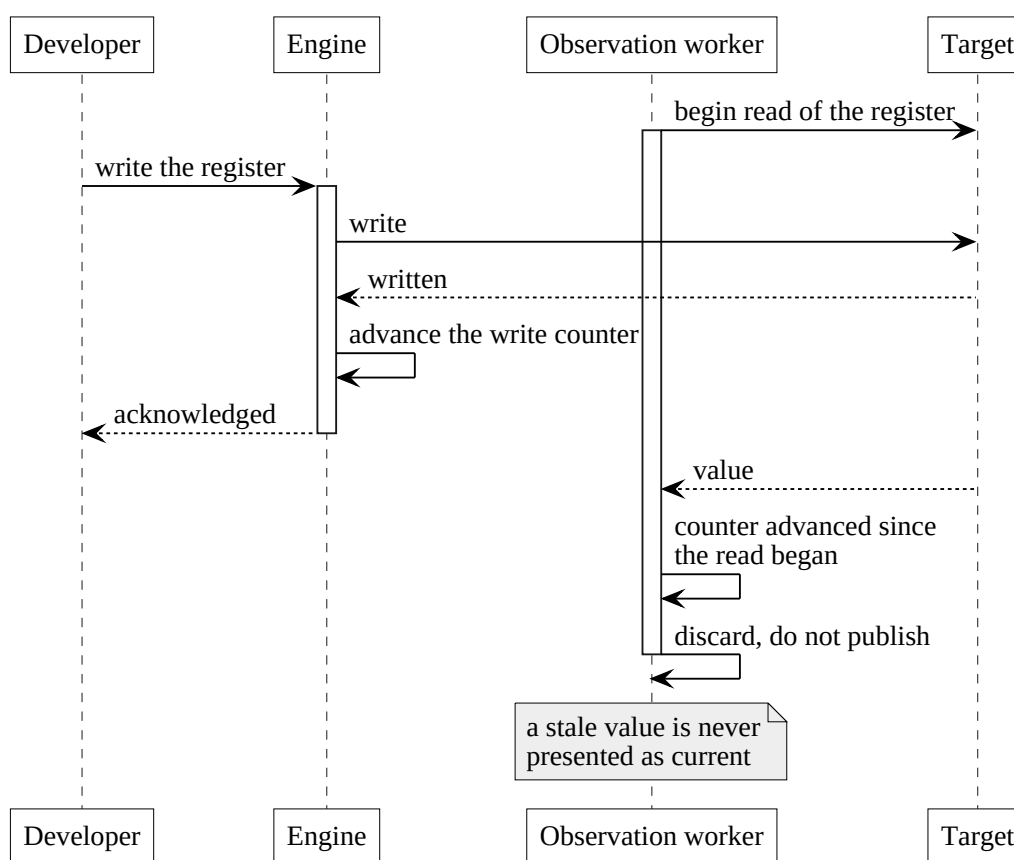


Figure 5.9: Peripheral write and observation

5.7.6 Variables and Expressions

Variables are obtained from LLDB. The engine converts each SBValue into the representation DAP defines, recursing into structured values on demand.

References to values are allocated from two spaces: one for values invalidated when the program resumes and one for values that persist. Separating them allows the first space to be cleared on every resumption

without disturbing the second.

Expression evaluation is exposed through the same component. A prefix character distinguishes an expression in the source language from a command addressed to LLDB.

5.7.7 Protocol Extensions

The protocol layer implements the message framing, the dispatch and the handler hierarchy of Section 4.2. The engine registers a handler for each DAP request listed in Table 7.1.

Handlers share a base that parses arguments, invokes the handler body and converts a returned error into an error response. Two handler shapes exist: one answering immediately and one deferring its response. The deferred shape is required by requests that must not be answered until configuration is complete.

Custom DAP requests expose embedded-specific functionality:

- Trace status and control
- Memory observation start and stop
- Peripheral enumeration and access
- Device reset

Custom events carry what the engine produces asynchronously, trace increments, memory view values, the values of a peripheral observation, and the report that the device was reset.

5.8 The Frontend

The frontend is a VS Code extension. It contributes the views that the standard DAP protocol does not cover. The extension contains no debugger logic. It presents data received from the engine.

Two views present trace. One groups the record by function and line. The other lists executed instructions in order. Both render source locations as links, so selecting an entry opens the corresponding source position. Discontinuities appear as explicit separators. Chapter 6 shows both views populated by a live capture.

A memory view and a set of live memory panels present regions of memory. A peripheral view presents the device description, with registers expanded to their individual fields.

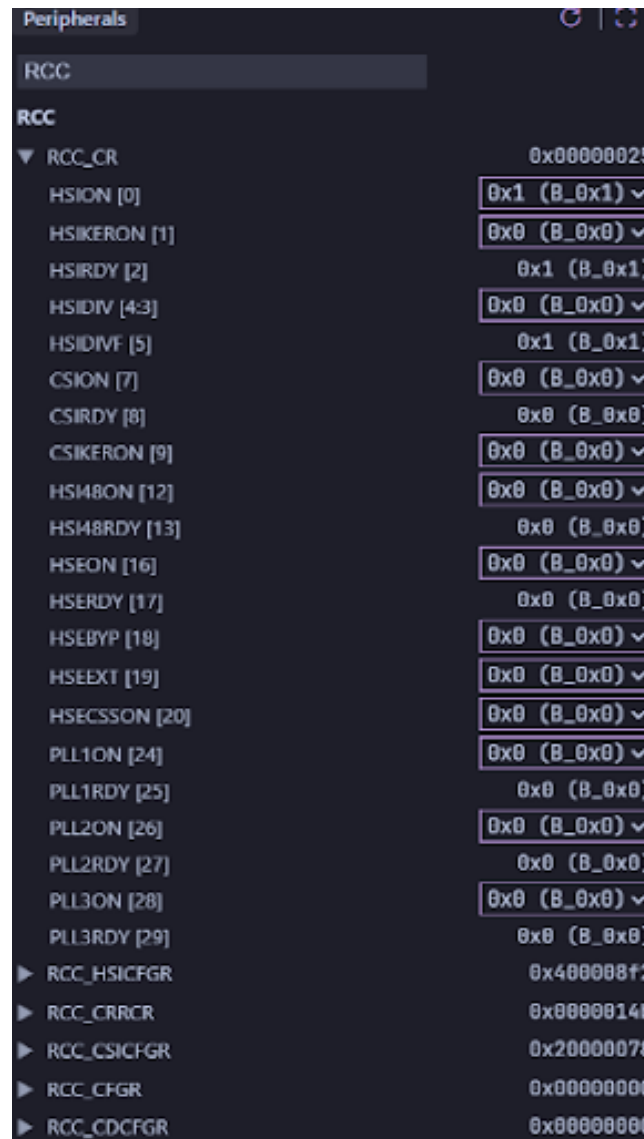


Figure 5.10: The peripheral view, with a register expanded to its individual fields.

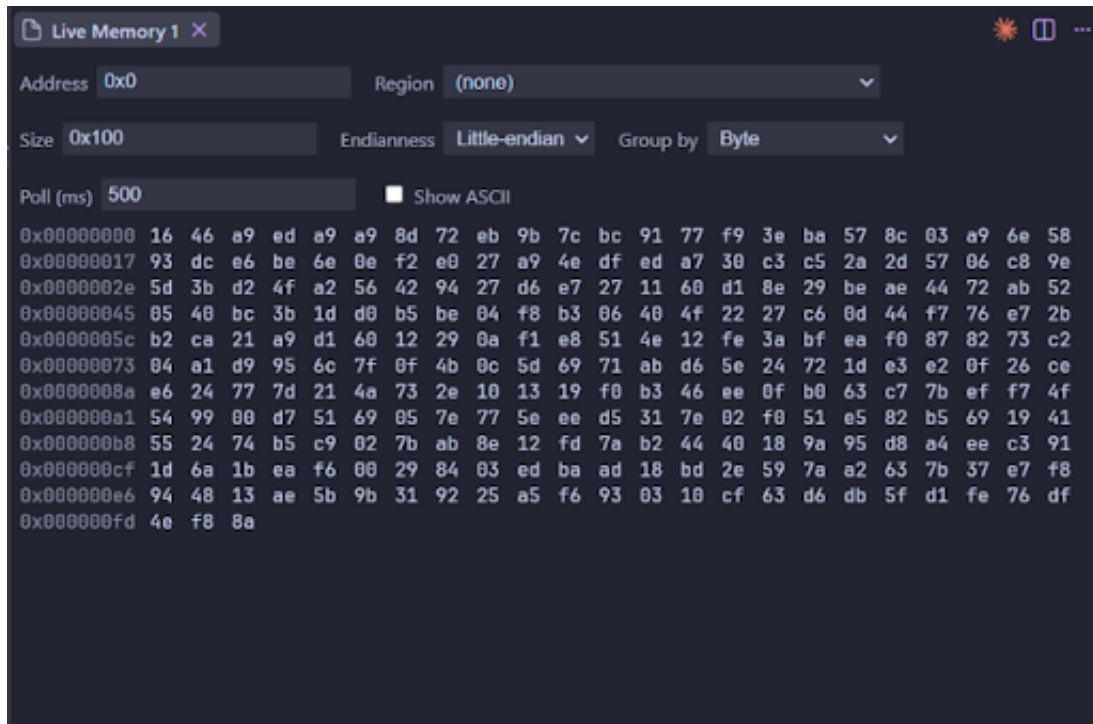


Figure 5.11: A live memory panel.

5.8.1 Portability and Provenance

The engine builds on Linux and on Windows. Portability work concentrated on protocol message reading (which requires a different mechanism on each platform), path resolution relative to the executable, and initialization requirements of the libraries used. An installer is produced for Windows, carrying the engine, its resources and the scripts of OpenOCD.

The engine is built on existing open-source work. The protocol layer began as the debug adapter distributed with LLDB and was restructured into the layers of Section 4.2. OpenOCD is a fork with the two CoreSight drivers and the GDB RSP corrections added. Decoding is performed by OpenCSD. Written for this project are the two CoreSight drivers, the trace pipeline from extraction through reconstruction, the domain components that constitute the core of the engine, the device description provider, and the frontend extension. The licenses of the adopted work are carried unchanged.

5.9 Conclusion

This chapter has described the realization of the three deliverables. The application note assembles into one document the procedure for using instruction trace on these devices. The trace pipeline configures the components on the device, extracts a capture over the ordinary debug connection, decodes it against the program image, and reconstructs the executed instructions with their source locations. The debugger makes that pipeline usable during development and supplies the conventional capabilities alongside it.

6 Experimental Validation and Results

Introduction

The earlier chapters set the goals, the design, and the code that met them. This chapter shows how the result was tested and what the tests proved. It describes the checks that run without the board, the checks that run on the board, and the checks that run through the VS Code extension. It then compares the reconstructed trace against three commercial debuggers, lists the bugs that appeared only on real hardware, gives the result against every requirement, and states what is left undone.

6.1 How the Engine Was Tested

The engine was tested in groups, set apart by what each one needs to run.

The first group needs only the compiled engine. It checks the parts that work on saved data: the protocol, the device-description parser, the disassembler, the trace decoder, and the reconstruction. These run on any machine, on every build, so a broken change is caught at once and the board is kept for the checks that truly need it.

The second group needs the board. A Python script sends the engine real debug-adapter requests over its input and reads the replies, so the capture path and the ordinary debug operations are checked against real hardware with no person watching.

The third group needs the board and the VS Code extension. It drives the extension and checks what the developer sees on screen.

Figure 6.1 lists the three groups and what each one checks.

The decoder and the reconstruction were written to run on a saved capture with no board attached, as Chapter 4 describes. A capture taken once becomes a test file that is replayed as often as needed. This is why the whole trace pipeline, apart from taking the capture itself, is tested without hardware.

Each test decides its own result. A test in the first group is a small program that compares what it produced against a value written into the test, then returns pass or fail. A script in the second group passes when every reply and event it expects arrives, in the right order, with the right content. A check in the third group passes when the view on screen matches what the engine sent.

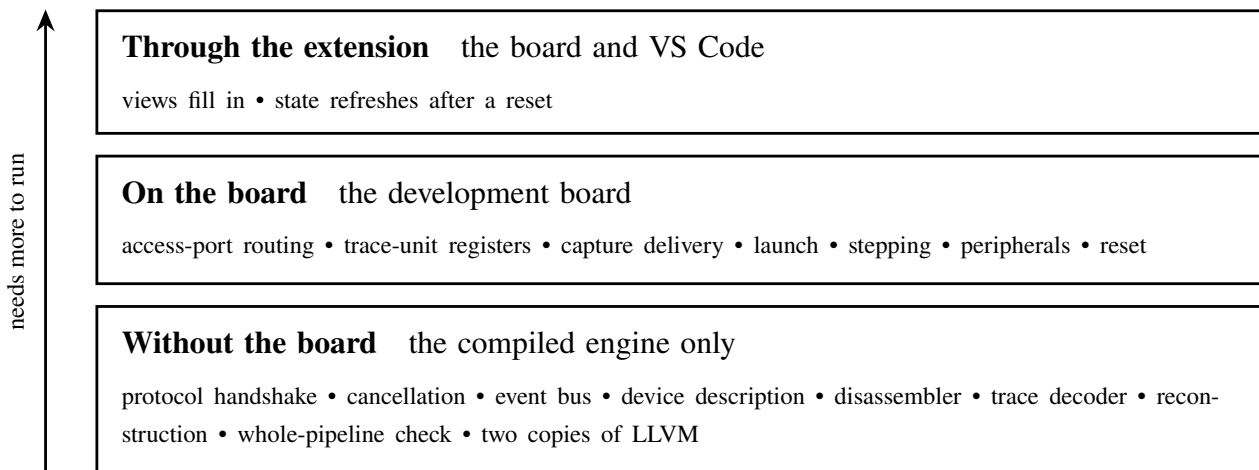


Figure 6.1: The three groups of tests and what each one checks

No test framework is used. Each off-board test is a small program that checks its own output and returns a code, which the build runs directly. This keeps the engine free of a dependency it would carry only for testing. Table 7.2 in the annexes lists the tests.

6.2 Hardware Test Bench

The bench is a NUCLEO-H7S3L8 board. It carries an STM32H7S3L8, a Cortex-M7 device, and an on-board STLINK-V3EC probe, joined to the host by a single USB cable. The board has no trace probe and no wire to the trace pins. The on-chip sink is a Trace Memory Controller holding two kilobytes. Every trace result below was taken over the same cable used for ordinary debugging, which is the goal set in Chapter 3.

The program under test is a small firmware project, built with debug information and with the optimization a developer would normally use.

Every result in this chapter was taken in a session the engine started itself, since trace, live memory reading, and peripheral reading all need the direct path to the device.

The same pipeline was also run on an STM32N6, which carries a newer trace unit on a Cortex-M55 processor. Only the base addresses and the configuration script changed. Neither driver was touched, because each driver reads what its component can do from that component's own identification registers. The result is therefore not tied to the one device it was measured on.

6.3 Tests That Run Without the Board

These tests run on saved data: one recorded trace capture, and firmware ELF files kept as fixtures. Each test drives one part of the engine and checks one result.

6.3.1 Protocol Handshake and Cancellation

The engine and the editor exchange JSON messages, each prefixed with its byte length. The first test writes a real `initialize` message to the engine's input and reads the reply. It checks that the reply is a proper `initialize` response and that it advertises trace support. Passing this means the engine frames a message, parses it, routes it to the right handler, and answers, all without a board and without the debugger library.

Cancelling a request was harder to get right, so it has its own test. The engine reads incoming messages on one thread and runs them on another, as Section 4.2.8 describes. The test sends a slow request that blocks, then a second request behind it, then a cancel for the second one. It checks that the second request comes back marked cancelled and that its handler never ran. This holds only if the reading thread can record the cancel while the running thread is still stuck on the slow request. The test proves the two threads are truly separate.

6.3.2 Event Delivery From Several Threads

Output is produced by several threads at once: the thread running debug requests, the thread receiving events from the debugger library, and the thread running OpenOCD. All of them hand their output to one object, the event bus, which passes each item to a single writer. If the event bus lost or duplicated an item under this load, output would be corrupted.

The test runs three threads that publish on the event bus at the same time, 2000 items each, and counts what comes out with a plain counter that has no lock of its own. The count comes out right only if the event bus already holds its own lock while it delivers. The same test runs under a thread sanitizer, which flags a data race the moment it happens. Running the whole engine under that sanitizer against the board is how the lock-ordering bug of Section 5.4 was found.

6.3.3 Static Program Analysis

Before a trace can be turned back into instructions, the engine reads the firmware ELF once and records, for every address, the instruction bytes, its mnemonic and operands, whether it is a branch, a call, a return or an indirect jump, and the source file, line and inlined functions it came from. Every later stage reads these facts by address.

The test checks this against three ELF files. The first has no debug information. On it the test confirms that a `bl` is marked a direct call with its target address filled in, that a `bx lr` is marked a return, and that asking for an address in the middle of an instruction returns nothing. The second is compiled with inlining forced on. On it the test confirms that one address unfolds into its full inline chain, for example an instruction inside an inlined `add_one` reported as `add_one` called from `compute` at the exact line. The third is the real firmware from the board. On it the test confirms that the pass, guided by the mapping symbols of Section 2.3.2.3, skips a

block of constant data sitting inside the code, a literal pool that runs from address `0x0800124C` to `0x08001264`, so those bytes are never disassembled as instructions. The test confirms that a real instruction just before the pool, at `0x08001214`, is decoded, and that the first word of the pool is not. Both addresses were read from the firmware’s symbol table by hand before the test was written.

6.3.4 Trace Decoding and Reconstruction

Decoding turns the captured bytes into trace elements. Reconstruction turns those elements into the list of executed instructions, each tied to a source line. Figure 6.2 shows the path and the test that guards each step. The element kinds are the ones in Table 2.5.

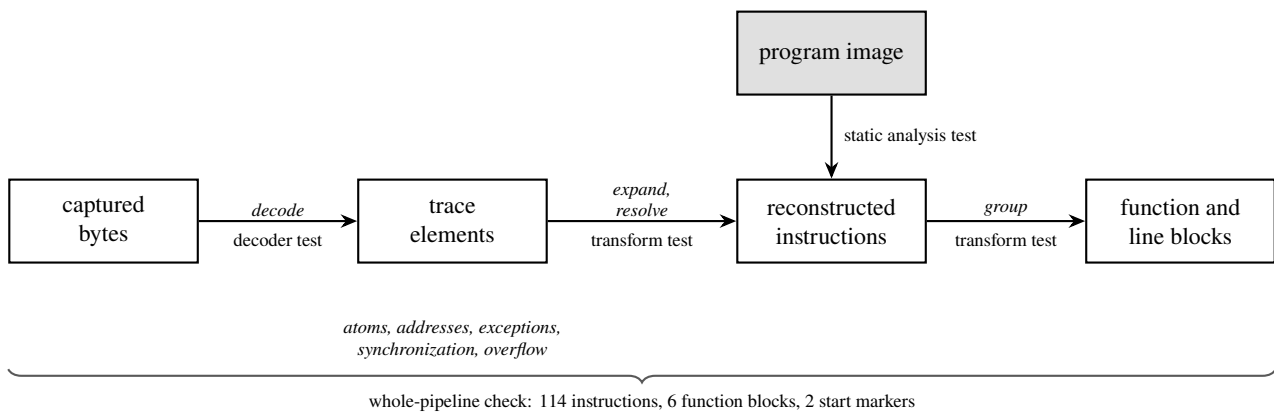


Figure 6.2: The trace path off the board, and the test that guards each step

The decoder is tested on its own. The test feeds it a saved capture together with the exact ETMv4 register values read from a real trace unit, and confirms the bytes decode all the way through without the decoder rejecting them. No firmware is given, so this test checks only that the bytes become elements.

Reconstruction is tested on a hand-built instruction table, with no real firmware and no decoder. This works because reconstruction reads plain value types that depend on neither library. One test builds three instructions, an `add`, a `bl` and a `bx`, feeds a range whose closing branch was not taken, and confirms that only the last instruction of that range is marked not-executed and that the mark does not leak into the next range. Another builds an inlined call chain and confirms that instructions from the same inlined line merge into one block, that instructions from different call chains stay in separate blocks, and that instructions in an assembly file never merge even when they share a line.

The whole pipeline is tested on one saved capture. The capture, the firmware, and the trace-unit registers are kept as test files. The test runs them through and checks the result against numbers frozen on the first correct run: 114 executed instructions, 6 function blocks, and 2 start markers. It also checks that every reconstructed address exists in the firmware and that at least one block has a real function name. These are the numbers reported in Table 6.5.

6.3.5 Parsing the Device Description

The peripheral views come from parsing the vendor's SVD file, an XML description of every peripheral on the chip. The test parses the file for the board's device and checks the result against fixed counts: 105 peripherals, 5 902 registers, 25 136 fields, and 30 490 named field values. It confirms that a peripheral defined as a copy of another, such as ADC2 from ADC1, still gets its own base address and its parent's registers. It checks the exact bit position and width of one named field. It confirms that a register cleared by the act of reading it is marked, so a view never reads it by accident. The test also records how much memory the parser uses, since one device's description is large.

6.3.6 Two Copies of LLVM in One Process

The engine links two copies of LLVM: one inside the LLDB library, and one behind the disassembler. Two copies in one program can clash at startup, for example both trying to register the same command-line option. A build that only links proves nothing, because the linker pulls in just the code that is actually called. The test therefore calls into both, in the same order the real program does, and confirms that both start up and that the disassembler opens a firmware file. This catches the clash at startup, ahead of any debug session.

6.4 Tests That Run on the Board

These tests need the board. Each one checks something only real hardware shows: how the debug access port behaves, how the trace components behave, and how the processor stops and steps. A Python script drives the engine over the debug-adaptor protocol, so the checks run with no person watching.

6.4.1 Reading Memory Through the Right Access Port

The engine reaches memory through a chosen access port, and this chip has two: one for the processor and one for direct memory access. The test reads the same address through each and confirms the two differ, because on this chip they genuinely see that address differently. It confirms that a read with no port named gives the processor's view, the default. It reads the same port twice and confirms the bytes match, which rules out a stale or a random result. It confirms that naming a port that does not exist fails. Together these show the port selection actually reaches the port asked for. The difference between the two ports was confirmed separately with OpenOCD's own `mdw` command.

6.4.2 Trace-Unit Registers and Capture Delivery

The decoder has to be set up with the trace unit's exact settings, so the values the engine reads from the unit are checked first. The test reads all of the trace unit's identification registers and compares each against a saved

reference, and confirms that naming a trace source that does not exist fails. It checks that the reported core name is Cortex-M7.

Capture is checked in the same test. It turns on the trace source and the sink, runs the processor for a moment, halts it, and records the callbacks the engine receives. It confirms exactly two callbacks for each halt: a 16-byte synchronization marker that the extraction driver inserts, then the captured bytes. The marker is checked byte for byte. Section 5.3.2 relies on this two-callback rule.

6.4.3 Launch and the Live Trace Path

The engine starts its own debug server, OpenOCD, inside its own process, rather than talking to a separate one. The test confirms that no separate OpenOCD process exists while it runs.

The same test runs the trace path in a real session. Figure 6.3 shows the steps. It turns on trace, sets a breakpoint at `main.c:106`, and lets the program run. When the program hits the breakpoint, the engine drains the trace and sends it. The test confirms that the trace arrives before the stop is reported, that its instructions carry real source lines, and that one block is named `main`. It runs the program a second time and confirms that the second batch of trace continues from where the first one ended, so the record grows across stops instead of starting over. Section 4.3.3 describes that accumulation.

6.4.4 Stepping

Stepping has to stop, and it must never freeze when the chip runs out of hardware comparators. The test steps over a call, into a call, and out of a call, and requires each one to stop within a set time. Stepping into `HAL_Init` has to leave the top of the stack inside `HAL_Init`, and stepping out has to leave it back in `main`. The time limit is what turns a freeze into a failing test instead of one that hangs forever. The bug where a step did not stop, listed in Section 6.7, was found here. Its cause was that LLDB placed its internal step breakpoints in flash as software breakpoints, which never fire on this chip.

6.4.5 Peripherals and Live Watching

The test reads the device model the engine built, reads port `GPIOA`, then writes and reads back a scratch register in the backup domain, chosen because writing it changes nothing on the board. It starts watching a peripheral and confirms that the frames arrive with a strictly rising sequence number. While the watch runs, it writes a register and confirms that no frame carries a value read before that write finished. Figure 5.9 shows how that race is avoided.

6.4.6 Reset and Resynchronization

Resetting the chip stops the processor by a path LLDB does not see, so afterward LLDB's picture of the program is stale. The resynchronization of Section 5.7.2 repairs this. The test resets the board, confirms the session reports a stop, then confirms that the stack and the variables reported afterward describe the program now running and not the one before the reset. The bugs where a restart left a stale view, listed in Section 6.7, were found here.

6.5 Tests Through the Extension

These checks cover what the developer sees in VS Code. Some run in the editor's test host with no board, checking the code that turns an engine message into a view. The rest run against the board and confirm the views fill in during a real session.

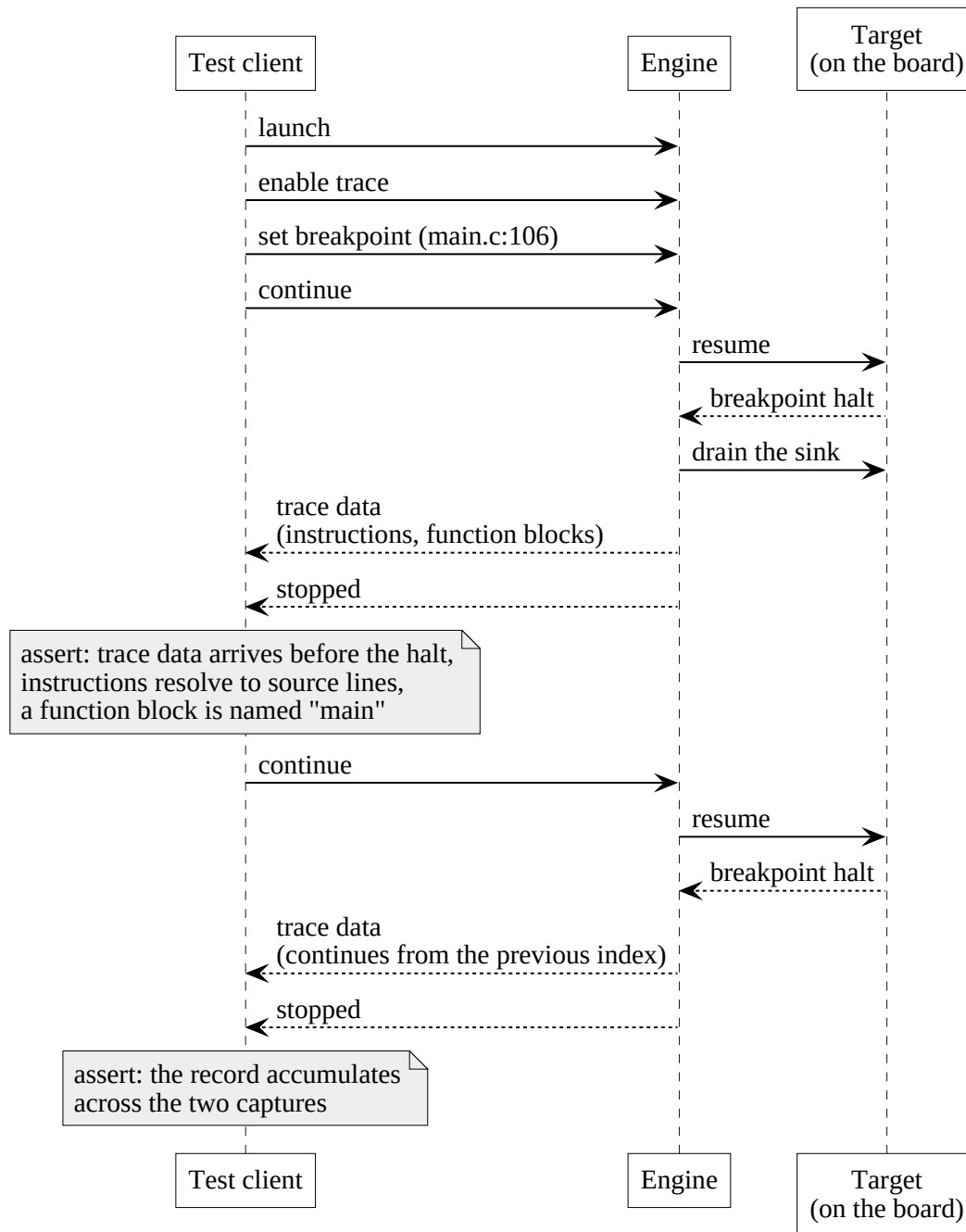
6.5.1 View Logic

Part of the extension is plain logic that a wrong result would corrupt without warning, so it is tested with no board. For the trace views, the test confirms that a new batch of trace is added to the list, that a reset clears the list, and that a gap shows as a plain separator and not as a fake instruction row, which keeps the two sides of a gap apart as Section 4.3.3 requires. It confirms that a small reducer turns each state the engine reports, idle, capturing, decode error, and the rest, into the right message on screen.

The peripheral view and the memory view carry bit arithmetic that a wrong result would corrupt silently. The test checks reading and writing one field without disturbing the bits around it, masking a value too wide for its field, and building the mask of a field that fills a whole 32-bit register without overflow. For memory it checks address parsing, grouping bytes into 16-bit and 32-bit words in both byte orders, and dropping a leftover partial cell.

6.5.2 End to End on the Board

The rest run against the board through the extension's real code path. One confirms that a batch of trace produced in a live session reaches the extension and fills both trace views. One confirms that launching starts the engine's own in-process OpenOCD, attaches, and resolves a breakpoint on a source line. One confirms, through the same path a developer uses, that stepping over a call stops instead of hanging. Figures 6.4 and 6.5 show both trace views filled from a live capture, the function view grouping the record by function and line, and the instruction view listing the executed instructions in order.

**Figure 6.3:** The live trace check driven against the device

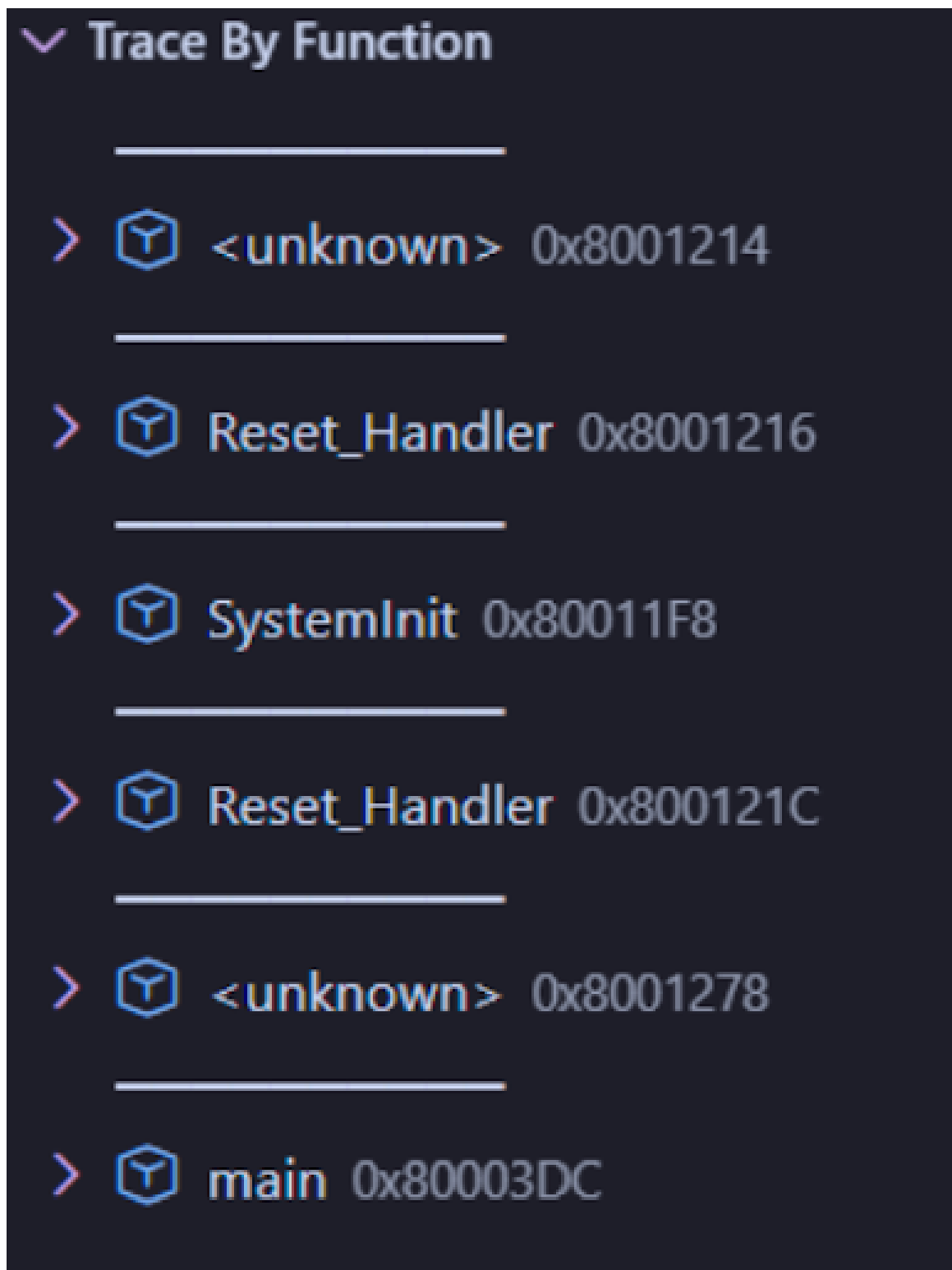


Figure 6.4: The function trace view, filled from a live capture. The record is grouped by function, each instruction carrying its resolved source location.

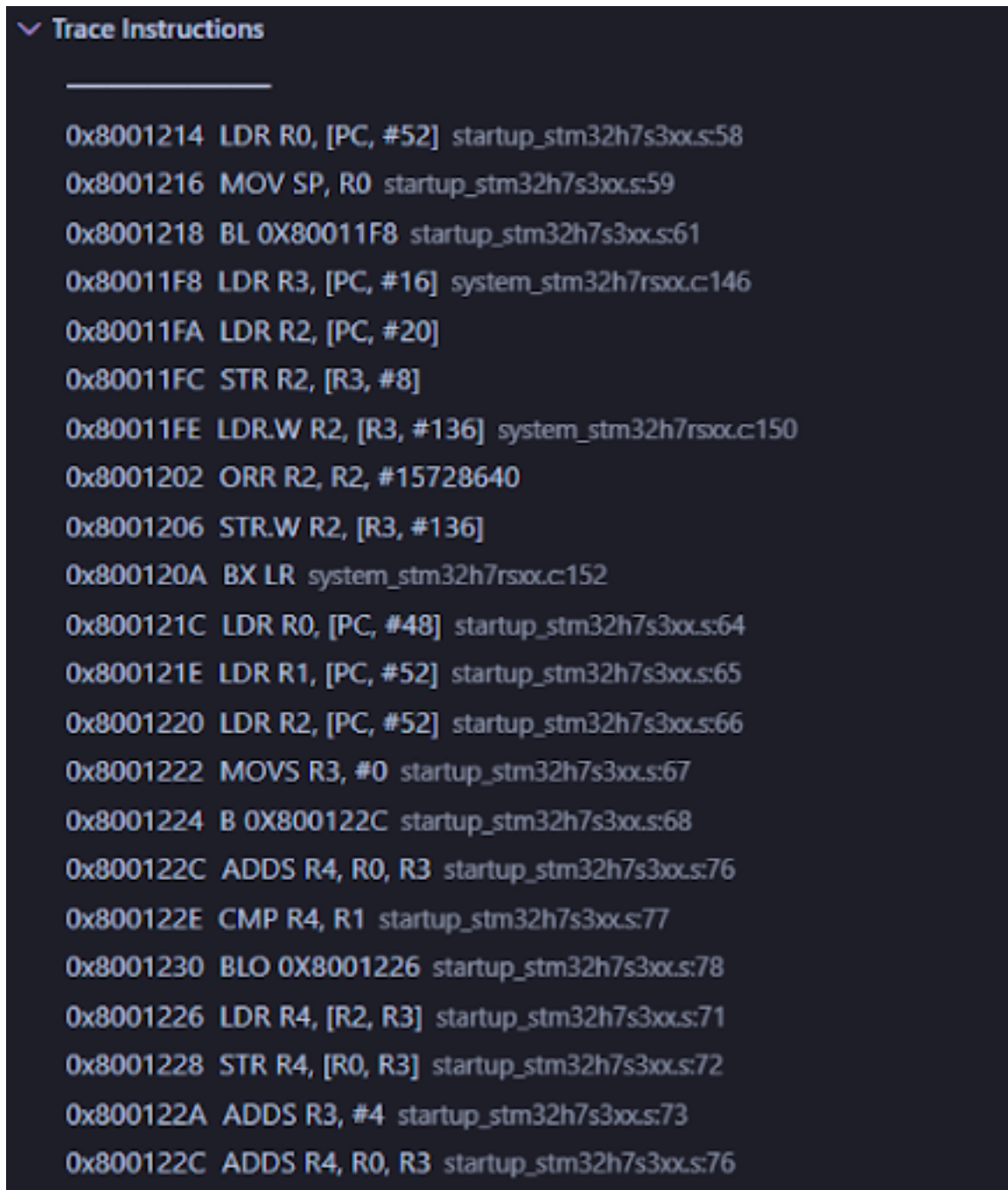


Figure 6.5: The instruction trace view, filled from a live capture. Executed instructions are listed in order, each carrying its resolved source location.

6.6 Comparison Against Other Tools

The tests above check the pieces. To check the reconstruction as a whole, its output was compared against tools built by other people. Table 6.1 lists the four comparisons.

The stored-capture check and the device check are the two tests already described in Sections 6.3.4 and 6.4. The other two compare against commercial tools.

The reconstructed instruction list was compared, line by line, against three commercial debuggers, each using its own vendor's probe. The lists matched. The only differences were in how each tool printed them.

A capture taken with a different vendor's probe was decoded by the same pipeline. After the framing is removed, the bytes were identical to those drained from the on-chip sink, which shows the pipeline does not depend on the probe that took the capture.

Table 6.1: The four comparisons and their outcomes

Comparison	What was compared	Outcome
Stored capture	The reconstructed instruction count against the frozen numbers, and every address against the disassembler	Counts match, every address resolves
Device	The trace-unit registers against the saved reference, and the callbacks on a halt	Registers match, two callbacks per halt
Commercial debuggers	The instruction list against three vendor debuggers, line by line, each on its own probe	Lists match, differences only in printing
Other vendor's capture	The byte stream against the block drained from the on-chip sink	Identical once the framing is removed

6.7 Defects Found on Hardware

Running the engine against a real board turned up bugs that no test on saved data would have caught. These bugs are why the on-board tests exist. Table 6.2 gives each one, its cause, its fix, and the section that describes the mechanism.

6.8 Results

The goal set in Chapter 3 was met. Instruction trace is captured from an on-chip sink, read out over the ordinary debug cable, decoded, reconstructed into the list of executed instructions, tied to source locations, and shown inside a development tool during a live debug session. No trace pins and no special probe are used.

The debugger that carries this also provides breakpoints with automatic choice between software and hardware, stepping by source line and by instruction, variable and expression reading, memory reading and writing, disassembly, device reset, and the added views for trace, live memory, and peripherals. The engine builds and runs on two host platforms.

Table 6.2: Defects found on hardware

Symptom	Cause	Fix	Section
A breakpoint on a source line resolved into a vendor header	Several source locations overlap at an inlined call site and the innermost was selected	Walk outward to the outermost inlined block and use the call site recorded there	5.7.3
Breakpoints could not be set in assembly files	The frontend had not declared that it supports breakpoints in that language	Declare the language	5.7.3
A restart left a stale view of the program	The reset is performed outside the conventional path, so LLDB was never told	The resynchronization mechanism	5.7.2
Variables and scopes were stale after a restart	The same cause	The same mechanism	5.7.2
Trace was inconsistent and often empty	The components were not armed on every transition that resumes the processor, and reset was mishandled	Couple both drivers to the run state of the target	5.3.2
Stepping did not halt where expected	The internal breakpoints LLDB places for a step were software breakpoints in flash, which never trigger	Force those breakpoints to use comparators	5.7.3

Every requirement is tied to the component that provides it and the section that builds it. Table 6.3 gives that mapping, and Table 6.4 gives how each requirement was checked and where the check runs.

Table 6.3: Requirements traced to components and their realization

ID	Component	Realization
FR1	Extended capabilities provider	CoreSight drivers
FR2	Instruction-trace manager	Run-state coupling
FR3	Extended capabilities provider	Trace extraction
FR4	Trace provider	Decoding
FR5	Trace provider	Reconstruction
FR6	Trace and disassembler providers	Static program analysis
FR7	Trace data model	Reconstruction
FR8	Frontend and protocol handlers	The frontend
FR9	Debug primitives provider	Breakpoints, session lifecycle
FR10	Memory manager	Memory and observation
FR11	Device provider	Peripherals
FR12	Disassembler provider	Variables and expressions
FR13	Session manager	Reset and resynchronization
NFR1	Hardware bench	Hardware test bench
NFR2	Engine build	Portability and provenance
NFR3	Frontend	The frontend

The barrier is inserted by the extraction driver ahead of every payload, so it never sat in the sink and is counted apart from the trace the device produced. Both figures are given because the first is what the buffer held and the second is what the decoder read.

The last row answers what a buffer holds. At the density of this capture, the two kilobytes of the sink

Table 6.4: Verification of the requirements

ID	How it was checked	Runs	Status	
FR1	Trace source and sink found and their registers read back on the board	board	Met	
FR2	Two callbacks seen on every halt	board	Met	
FR3	Every result in this chapter taken over the debug cable alone	board	Met	
FR4	The saved capture decoded all the way through, Figure 6.2	no board	Met	
FR5	Every reconstructed address exists in the firmware, the counts match the frozen numbers of Table 6.5, and the list matches three vendor debuggers, Section 6.6	both	Met	
FR6	Function blocks carry resolved names in the same check	no board	Met	
FR7	Gaps counted and shown as separators in the same check	no board	Met	
FR8	Trace views filled during a live session, seen in the editor	extension	Met	
FR9	Stepping over, into and out of a call, and breakpoints, on the board	board	Met	
FR10	Memory reading and register access on both paths, on the board	board	Met	
FR11	Peripheral reading and writing on the board	board	Met	
FR12	Disassembly, scopes and variables on the board	board	Met	
FR13	Reset then the resynchronization of Section 5.7.2, on the board	board	Met	
NFR1	The bench carries no trace probe and no wire to trace pins, Section 6.2	board	Met	
NFR2	The engine builds and runs on both host platforms	no board	Met	
NFR3	The frontend is an extension speaking the standard protocol	extension	Met	

hold on the order of eighteen hundred executed instructions. The figure is a guide only, since the density depends on how often the program branches.

6.9 Limitations and Future Work

The work meets its goals within limits, and those limits point to the next steps. The limits fall into two groups. Table 6.6 gives the limits of the trace pipeline, the part that turns a capture into a reconstructed program. The limits of the engine that carries the pipeline follow the table, with the directions that extend the work.

Table 6.5: Measured figures for captured traces

Quantity	Value
Interval traced	Reset handler to the entry of main
Trace produced by the device	128 bytes, eight formatter frames
Host-inserted synchronization barrier	16 bytes, one frame
Bytes presented to the decoder	144 bytes, nine formatter frames
Elements retained	35
Executed instructions reconstructed	114
Function blocks	6
Start-of-capture markers	2
Discontinuities within the traced execution	0
Executed instructions per byte produced by the device	0.89

Table 6.6: Limitations of the trace pipeline

Limitation	What it means
Component discovery	The trace source and sink are found by name among the objects the configuration creates, and their base addresses are stated there. A new device needs a configuration file written for it.
One protocol version	The drivers and the decoder handle the version the devices used implement. No device reachable by the on-chip route is lost, since every STM32 device with an on-chip trace sink implements that version, as Table 1.3 states.
One source and one sink	The first of each is used, so a device with several is not fully exploited. Tracing several cores is outside the scope of a proof of concept on a single-core target.
Timing discarded	Timestamps and cycle counts are decoded and then dropped, so no timing analysis is possible without changing that stage.
Exceptions not shown	The elements are kept and no view presents them, so an exception shows in the record as a jump in address.
No trace filtering	Tracing runs unfiltered, so the amount of trace is bounded only by the size of the buffer. On these devices filtering would be driven from the processor comparators.
Return classification	LLVM's analyzer does not recognize the instruction forms this architecture uses for a return, so those forms are matched on the printed text.
Off-board decoding not exposed	The stages need only a capture, a firmware image and the saved registers. The standalone entry point that offered this during development was not carried forward.

Further limits concern the engine. The two disassembly paths have not been merged. A few requests drive LLDB outside the serialization of Section 5.4. The capabilities that need the direct path are missing when attaching to an existing session. The frontend is not packaged for distribution.

Several directions extend the work. Keeping the timing elements now dropped would allow profiling from the same captures. Showing exceptions directly would complete the picture across interrupt entry. Turning on the trace unit's filtering would let the buffer cover a chosen region. Reading the component layout from the ROM tables would remove the per-device configuration file.

6.10 Conclusion

The result was checked against the goals of Chapter 3 and the requirements taken from them. The parts were tested without the board wherever the board was not needed, the capture and the stepping were tested on the board, and the reconstructed instruction list was matched line by line against three commercial debuggers. Every requirement was met. Instruction trace was captured over the ordinary debug cable, with no trace pins and no probe beyond the one on the board. The limits that remain point to the next steps.

Conclusion

The capability this project set out to reach was already present in the hardware. Instruction trace is implemented across a large part of the STM32 range, and on the families carrying a Trace Memory Controller a capture may be taken without leaving the chip. What was missing was a way to reach it with the tools and the probe an ordinary debug session already uses.

The work produced three results. An application note assembles into one document the procedure for using instruction trace on these devices, which the architecture specifications and the reference manuals state separately. A proof of concept demonstrates that a capture may be obtained from an on-chip sink, decoded against the program image, and reconstructed into the instructions that executed. A debugger carries that capability into a development tool, alongside breakpoints, variable inspection, memory and peripheral views, and disassembly.

Section 6 states what the result achieves. A capture of 128 bytes taken over the ordinary debug connection reconstructs 114 executed instructions, each attributed to a function, a source file and a line. No trace pins are used, and no probe beyond the one embedded on the development board.

Two conclusions carry beyond the project. The first is that the barrier to instruction trace on these devices was never the hardware. It was the distance between what the documentation states and what a developer must do, together with the absence of an open implementation to close that distance. The second is that reconstructing execution is not a decoding problem alone. The trace reports which branches were taken and nothing further, so the program image supplies everything else, and the result is only ever as good as the correspondence between the two.

What remains is stated where it belongs. The pipeline captures without filtering, discards the timing elements it decodes, and reads the trace topology from a configuration file, where the device's own ROM tables could supply it. Each of these is a boundary of the implementation and not of the approach, and Section 6.9 states what closing them would require.

Those boundaries also mark the directions the work opens. Retaining the timing elements would bring profiling from the same captures. Presenting exceptions explicitly would extend the reconstruction across interrupt entry. Reading the component topology from the device would remove the last per-device configuration. Each of these builds on the pipeline this project established, and none of them revisits the barrier the project set out to remove, since that barrier is now gone.

Bibliography

- [1] Linaro Limited, *OpenCSD: An Open Source CoreSight Trace Decode Library*, Version 1.8.3. [Accessed August 2026]. [Online]. Available: <https://github.com/Linaro/OpenCSD>.
- [2] STMicroelectronics, *ST company presentation*, [Accessed July 2025], 2024. [Online]. Available: https://www.st.com/resource/en/company_presentation/company_presentation.pdf.
- [3] STMicroelectronics, *AN4989: STM32 Microcontroller Debug Toolbox*, Rev 4, STMicroelectronics, 2026.
- [4] STMicroelectronics, *RM0433: STM32H742, STM32H743/753 and STM32H750 Advanced Arm-based 32-bit MCUs Reference Manual*, Rev 8, STMicroelectronics, 2023.
- [5] STMicroelectronics, *RM0477: STM32H7Rx/7Sx Arm-based 32-bit MCUs Reference Manual*, Rev 9, STMicroelectronics, 2025.
- [6] IAR Systems, *Requirements for Using ETM Trace, IAR Embedded Workbench for Arm*, [Accessed September 2026]. [Online]. Available: <https://docs.iar.com/ewarm/10.1x/en/c-spy-debugging/trace/introduction-to-using-trace/requirements-for-using-etm-trace.html>.
- [7] The OpenOCD Project, *Open On-Chip Debugger*, Version 0.12.0. [Accessed August 2026]. [Online]. Available: <https://openocd.org/>.
- [8] The pyOCD Project, *pyOCD: Python-based Debugger and Programmer for Arm Cortex-M*, [Accessed September 2026]. [Online]. Available: <https://pyocd.io/>.
- [9] The Linux Kernel Developers, *Perf Tool Support for Arm CoreSight*, [Accessed September 2026]. [Online]. Available: <https://docs.kernel.org/trace/coresight/coresight-perf.html>.
- [10] M. Matraszek, M. Banaszek, W. Ciszewski, and K. Iwanicki, “FrankenTrace: Low-Cost, Cycle-Level, Widely Applicable Program Execution Tracing for ARM Cortex-M SoC,” in *Cyber-Physical Systems and Internet of Things Week 2023 (CPS-IoT Week Workshops '23)*, ACM, 2023. DOI: 10.1145/3576914.3587521.
- [11] Z. Ning and F. Zhang, “Ninja: Towards Transparent Tracing and Debugging on ARM,” in *26th USENIX Security Symposium*, USENIX Association, 2017.
- [12] Microsoft, *Debug Adapter Protocol*, [Accessed August 2026]. [Online]. Available: <https://microsoft.github.io/debug-adapter-protocol/>.

- [13] Free Software Foundation, *Debugging with GDB: The GDB Remote Serial Protocol*, [Accessed August 2026]. [Online]. Available: <https://sourceware.org/gdb/current/onlinedocs/gdb.html/Remote-Protocol.html>.
- [14] The LLVM Project, *LLDB GDB Remote Protocol Extensions*, [Accessed August 2026]. [Online]. Available: <https://lldb.llvm.org/resources/lldb gdbremote.html>.
- [15] C. Markusson, "Implementation of an application debugger for software in embedded systems," Report no. LIU-IDA/LITH-EX-A--08/050--SE, M.S. thesis, Linköpings universitet, Department of Computer and Information Science, 2008.
- [16] DWARF Debugging Information Format Committee, *DWARF Debugging Information Format Version 5*, 2017. [Online]. Available: <https://dwarfstd.org/doc/DWARF5.pdf>.
- [17] Arm Limited, *ELF for the Arm Architecture*, AAELF32, ARM IHI 0044, Arm Limited, 2024. [Online]. Available: <https://github.com/ARM-software/abi-aa/blob/main/aaelf32/aaelf32.rst>.
- [18] Arm Limited, *CMSIS System View Description (CMSIS-SVD)*, [Accessed August 2026]. [Online]. Available: <https://open-cmsis-pack.github.io/svd-spec/main/index.html>.
- [19] Arm Limited, *Learn the Architecture: Introducing CoreSight Debug and Trace*, Issue 01, document 102520_0101_01_en, Arm Limited, 2025.
- [20] Arm Limited, *AMBA ATB Protocol Specification, ATBv1.0 and ATBv1.1*, ARM IHI 0032C, Arm Limited, 2012.
- [21] Arm Limited, *Arm Debug Interface Architecture Specification, ADIv5.0 to ADIv5.2*, ARM IHI 0031H, Arm Limited, 2022.
- [22] Arm Limited, *Arm Architecture Reference Manual, Armv7-M Architecture Profile*, Issue E.e, ARM DDI 0403E.e, Arm Limited, 2021.
- [23] Arm Limited, *Understanding Trace: Trace Overview*, Version 2.0, document 102119_0200_00, Arm Limited, 2020.
- [24] Arm Limited, *Arm Embedded Trace Macrocell Architecture Specification, ETMv4.0 to ETMv4.6*, ARM IHI 0064H.b, Arm Limited, 2023.
- [25] Arm Limited, *Embedded Trace Macrocell Architecture Specification, ETMv1.0 to ETMv3.5*, ARM IHI 0014Q, Arm Limited, 2011.
- [26] Arm Limited, *Arm CoreSight Architecture Specification v3.0*, ARM IHI 0029G, Arm Limited, 2022.
- [27] Arm Limited, *Arm CoreSight SoC-400 Technical Reference Manual*, Issue 09, document 100536_0302_09_en, Arm Limited, 2023.

- [28] Arm Limited, *Arm CoreSight Trace Memory Controller Technical Reference Manual*, Revision r0p1, ARM DDI 0461B, Arm Limited, 2013.
- [29] Arm Limited, *Arm Cortex-M7 Processor Technical Reference Manual*, Revision r1p2, ARM DDI 0489D, Arm Limited, 2018.
- [30] STMicroelectronics, *STM32H7S3x8, STM32H7S7x8 Datasheet*, DS14359 Rev 6, STMicroelectronics, 2025.
- [31] STMicroelectronics, *UM3276: STM32H7Rx/7Sx Nucleo-144 Board (MB1737)*, Rev 2, STMicroelectronics, 2024.
- [32] Arm Limited, *Arm CoreSight ETM-M7 Technical Reference Manual*, Revision r0p1, ARM DDI 0494D, Arm Limited, 2016.
- [33] The LLVM Project, *The LLVM Target-Independent Code Generator: The MC Layer*, LLVM 22. [Accessed August 2026]. [Online]. Available: <https://llvm.org/docs/CodeGenerator.html>.
- [34] The LLVM Project, *The LLDB Debugger*, LLVM 22. [Accessed August 2026]. [Online]. Available: <https://lldb.llvm.org/>.

Annexes

Annexe 1. Registered Protocol Requests

Table 7.1: Registered protocol requests by area

Area	Requests	Kind
Lifecycle	initialize, launch, attach, configurationDone, restart, disconnect, cancel	Standard
Execution	continue, next, stepIn, stepOut, stepInTargets, pause	Standard
Breakpoints	setBreakpoints, setFunctionBreakpoints, setInstructionBreakpoints, setDataBreakpoints, setExceptionBreakpoints, breakpointLocations, dataBreakpointInfo	Standard
Inspection	threads, stackTrace, scopes, variables, setVariable, evaluate, completions, exceptionInfo	Standard
Memory	readMemory, writeMemory	Standard
Disassembly	disassemble	Standard
Symbols	modules, moduleSymbols, compileUnits, locations, source	Standard and custom
Trace	trailerTraceEnable, trailerTraceDisable, trailerTraceStatus	Custom
Observation	trailerWatchStart, trailerWatchStop	Custom
Peripherals	trailerDeviceInfo, trailerPeripheralDetail, trailerPeripheralRead, trailerPeripheralWrite, trailerPeripheralWatchStart, trailerPeripheralWatchStop	Custom

Annexe 2. Test Inventory

Table 7.2: Test inventory

Test	Tier	What it verifies
device_svd_test	One	The device description parser against a vendor file
device_provider_test	One	The memory map and the peripheral model built from that description
program_disassembler_test	One	Spans, instruction decoding and source resolution
trace_decoder_test	One	A full decode of a stored capture without a fatal response
transform_test	One	Instruction expansion and function block grouping over a record sequence
trace_provider_test	One	The whole offline pipeline, against counts frozen from a verified run and against properties checked independently
event_bus_test	One	Publication on the event bus from several threads at once
orchestrator_cancel_test	One	The cancellation path through the dispatcher
llvm_lldb_coexistence_test	One	That the compiler library and the trace decoder link into one program
handshake_test	One	The protocol handshake and the capability set returned to the client
openocd_provider_hw_test	Two	Trace components located, register readback and capture delivery
launch_smoke_test	Two	Threads, call stack, memory reading, disassembly and variable inspection in a launched session
attach_smoke_test	Two	Threads and call stack in a session attached to a running server
stepping_smoke_test	Two	Stepping over, into and out of a call, and the behavior when the comparators are exhausted
peripheral_smoke_test	Two	Peripheral enumeration, register reading and writing, and an observation discarding a frame that overlaps a write

Abstract

This report describes a debugger that captures, decodes and presents the instruction trace of an STM32 microcontroller. Instruction trace records the path a program actually took through its code. The hardware producing it is present across much of the STM32 range, yet it is reachable only through proprietary tools and probes most developers do not have. Two CoreSight drivers were added to an open-source debug server, so that the trace unit and the on-chip trace buffer may be configured and drained over the ordinary debug connection. A pipeline decodes the captured bytes against the program image, reconstructs the executed instructions, attributes each to a function, a source file and a line, and presents the result inside a development tool during a live debug session. A capture of 128 bytes, drained from the on-chip buffer with no trace pins in use, reconstructs 114 executed instructions.

Keywords : Instruction trace, CoreSight, Embedded Trace Macrocell, Embedded debugging, STM32, Debug Adapter Protocol, OpenOCD.

Résumé

Ce rapport décrit un débogueur capable de capturer, de décoder et de présenter la trace d'instructions d'un microcontrôleur STM32. La trace d'instructions enregistre le chemin que le programme a réellement suivi dans son code. Le matériel qui la génère équipe une grande partie de la gamme STM32, mais il n'est exploitable qu'au moyen d'outils propriétaires et de sondes dont la plupart des développeurs ne disposent pas. Deux pilotes CoreSight ont été ajoutés à un serveur de débogage libre, afin que l'unité de trace et la mémoire tampon de trace intégrée au circuit puissent être configurées et vidées via la connexion de débogage habituelle. Une chaîne de traitement décode ensuite les octets capturés à l'aide de l'image du programme, reconstruit les instructions exécutées, rattache chacune à une fonction, à un fichier source et à une ligne, et présente le résultat au sein d'un environnement de développement, pendant une session de débogage. Une capture de 128 octets, extraite de la mémoire tampon intégrée au circuit et obtenue sans recourir aux broches de trace, permet de reconstruire 114 instructions exécutées.

Mots clés : Trace d'instructions, CoreSight, Embedded Trace Macrocell, Débogage embarqué, STM32, Debug Adapter Protocol, OpenOCD.